

YouDescribe iOS app -
An accessibility tool for adding audio descriptions to online videos

A Thesis submitted to the faculty of
San Francisco State University
In partial fulfillment of
the requirements for
the Degree

Master of Science
In
Computer Science

by
Rupal Khilari
San Francisco, California
May 2017

Copyright by

Rupal Khilari

2017

CERTIFICATION OF APPROVAL

I certify that I have read ‘YouDescribe iOS app – An accessibility tool for adding audio descriptions to online videos’ by Rupal Khilari, and that in my opinion this work meets the criteria for approving a thesis submitted in partial fulfillment of the requirement for the degree Master of Science in Computer Science at San Francisco State University.

Dr. Ilmi Yoon, Ph.D. Associate Professor

Dr. William Hsu, Ph.D. Associate Professor

YouDescribe iOS app –
An audio annotation/playback tool to make online videos accessible.

Rupal Khilari
San Francisco, California
2017

The aim of this project is to build an iOS application for ‘YouDescribe’. ‘YouDescribe’ is a project founded by Smith-Kettlewell Video Description Research and Development Center (VDRDC) with a view to enable and encourage sighted volunteers to describe online videos for the benefit of the blind and visually impaired. Their platform includes a server and a web-interface to facilitate this. Due to the increasing availability of online content through smartphones and the convenience it offers, there was a strong need to make YouDescribe available on mobile phones as well. Also, as YouDescribe requires complex interaction with audio functions, its web version was not available on mobile browsers. Stemming from these needs, the main objective of this project is to develop a native iOS application for YouDescribe in Swift 3.0 that provides an accessible, mobile application for playing back and recording audio descriptions for online videos (particularly YouTube). Using this app, sighted volunteers can record and upload audio descriptions onto online videos to make them more accessible. We believe that the YouDescribe iOS app will not only enable visually challenged users to enjoy videos on their iPhones, but also encourage volunteers to contribute more in making video content accessible to everyone.

I certify that the Abstract is a correct representation of the content of this thesis.

Chair, Thesis Committee

Date

ACKNOWLEDGEMENTS

Firstly, I'd like to sincerely thank my advisor, Dr. Ilmi Yoon for believing in me and giving me the opportunity to work on this project under her guidance. Her insights, enthusiasm, encouragement and support during the course of the project are very inspiring and helped me a lot.

My sincere thanks to Dr. Joshua Miele of the Smith-Kettlewell Eye Research Institute for trusting me to be a part of this amazing project founded by their organization. Also, for patiently answering my questions, testing every app release and providing extremely valuable and detailed feedback. I am grateful to Keith Williams for actively supporting the test and production servers during the app development phase. I also appreciate the wonderful first hand inputs I received from one of the first describers for YouDescribe, Dr. Yue Ting Siu, assistant professor in the Program for Visual Impairments at SFSU.

I'd like to thank the department Chair, Dr. William Hsu, for his advice and valuable inputs on this project, being a part of my thesis committee. His effective management of the Computer Science department helped me have a wonderful experience here. I am also grateful to the department office manager, Jen Schwartz for efficiently handling all the administrative help I needed during the project and throughout the Master's program.

Lastly, I'd like to thank my family for their love, support, guidance and encouragement whenever I needed it.

TABLE OF CONTENTS

List of Tables	viii
List of Figures	ix
List of Appendices	xi
1. Introduction	1
1.1. Objectives	2
1.2. Existing system	4
1.3. Roadmap	6
2. Requirements Analysis	7
2.1. Use cases	7
2.2. Terminology	10
2.3 Functional requirements.....	11
2.3.1. Scope	15
2.4 Non-functional requirements	16
2.5 User interface mockups.....	17
3. System Architecture and Design.....	20
3.1. High-level system architecture	21
3.2. Application architecture	25
3.3. Evaluation of libraries	26
3.4. Application Programming Interfaces (APIs)	33
3.4.1. DVX (Descriptive Video Exchange) API.....	34
3.4.2. New YouDescribe API (For Version 2).....	36
3.4.3. YouTube Data API	36
3.5. Sequence of data flow	37
3.6. Accessibility considerations.....	38
4. Implementation	45
4.1. Tasks and Milestones	45
4.2. Class Diagram	47

4.3. Code structure	49
4.4. UI Implementation and functions	56
4.5. Developing for Accessibility	70
4.5. Challenges faced	73
5. Testing.....	78
5.1. Unit testing.....	78
5.2. Usability testing	80
5.2.1. Usability Test plan and results	80
5.2.3. Accessibility testing	81
5.3. Quality assurance (QA) testing	84
5.4. Performance testing	87
5.5. Code review and linting	89
5.6. Continuous testing through TestFlight	90
6. Deployment.....	92
7. Future plans.....	94
8. Conclusion	97
9. References	98
10. Appendix.....	101

LIST OF TABLES

Tables	Page
1. Comparison of video players for iOS apps	30
2. Summary of DVX API endpoints	35
3. Summary of the new YouDescribe API (version 2) endpoints	36
4. Quality assurance tests and results	84

LIST OF FIGURES

Figures	Page
1. Architecture of the existing system	5
2. Mockup: Launch screen	18
3. Mockup: Home screen with video listing.....	18
4. Mockup: Search results and filters	19
5. Mockup: Player component.....	19
6. Mockup: Authoring component	20
7. Mockup: Authoring settings	20
8. MVC Architecture	26
9. DVX Database schema.....	34
10. Sequence diagram.....	38
11. iOS VoiceOver Gestures	40
12. Class Diagram	48
13. Code Structure	49
14. ViewController life cycle	55
15. Podfile for the YouDescribe app	56
16. Storyboard of the app	59
17. UI Screen: Home Screen	60
18. UI Screen: Detail menu options	60
19. UI Screen: Login	61
20. UI Screen: Register	61
21. UI Screen: User account on home screen.....	61
22. UI Screen: Accounts page	62
23. UI Screen: All Search Results	63
24. UI Screen: Filtered Search Results.....	63
25. UI Screen: List videos by describer	63
26. UI Screen: Video player	64
27. UI Screen: Video detail information	66
28. UI Screen: Social app integration, request for description.....	67

29. UI Screen: Detail option to Edit a description	68
30. UI Screen: Detail option to create a new description on YouTube video	69
31. UI Screen: Authoring (Recording) interface	70
32. UI Screen: DVX internal errors.....	75
33. UI Screen: Debugging with Charles Web Proxy.....	76
34. Unit Tests	79
35. Unit Tests for DVX API calls	79
36. Accessibility Inspector: Label, trait and hint.....	81
37. Accessibility Inspector: Run Audit	82
38. VoiceOver tests	83
39. Instrument Screenshot for App's performance	88
40. Code style violations using tailor	89
41. Correction of style violations	90
42. iOS app deployment	93

LIST OF APPENDICES

Appendix	Page
1. Example of a DVX API call.....	98
2. Example of a YouDescribe API (version 2) call	98
3. Example of YouTube Data API call	100

Chapter 1

Introduction

Digital communication through video has received a huge impetus over the last decade and has become a part of our daily lives. The blind or visually impaired often miss out on the visual information conveyed through videos. An effective way to bridge this gap to a good extent has been through adding audio narrations (descriptions) to these videos. Audio descriptions^[1] make feature films, television programs, and museum exhibits accessible to people who are visually impaired thus engaging them with the video content to a greater extent. Over the past 5 years, organizations such as the Media Access Group^[2], American Foundation for the Blind^[3], DVS (Descriptive Video Service)^[4] have paved the way to making television programs, feature films accessible to the blind. With the increasing

availability of online videos for education, entertainment and news available on YouTube^[5], Hulu^[6] and Vimeo^[7], there is a need to make this content accessible as well. From the available online video websites, YouTube is known to be the most popular with an estimated 1 billion unique visitors every month ^[67,68].

The Smith-Kettlewell Research Video Description Research and Development Center^[8] developed a system consisting of a database and API known as Descriptive Video Exchange (DVX)^[9]. DVX holds descriptive audio clips, synchronization and other information aimed to be used to describe any online media. ‘YouDescribe’ (www.youdescribe.org)^[10] is a web-based client for DVX that enables playback and recording of audio clips for YouTube videos. It follows a crowdsourcing model where any sighted volunteer can record and upload audio descriptions for YouTube videos upon request. These audio annotated videos can be shared across the internet, thus making them accessible to all.

1.1 Objectives

Due to the increasing availability of online content through smartphones and the convenience it offers, there was a strong need to make YouDescribe available on mobile phones as well. Statistics show that over half of video views on YouTube come from mobile devices ^[68]. Also, as YouDescribe requires complex interaction with audio functions, its web version was not available on mobile browsers. Due to YouDescribe’s use of a flash-based player for playback and the requirement of permissions to access the

device's microphone, these key YouDescribe components are not available on mobile browsers. Stemming from these needs, the main objective of this project is to develop a native iOS application for YouDescribe that provides an accessible, mobile application for playing back and recording audio descriptions for online videos (particularly YouTube). The iOS^[11] ecosystem has historically included built-in support for accessibility through VoiceOver. VoiceOver is a screen reader that allows its users with visual impairments to hear out loud what is displayed on the screen using a few simple gestures. As iOS is very popular among visually challenged users, it was the preferred platform for development. The key technical outcomes from our project are the native iOS YouDescribe app for multiple devices, design approaches for building accessible components in Swift 3.0^[12], and potential future extensions. Since YouDescribe heavily relies on volunteer efforts, its usage will receive an impetus with a mobile client. Users will be able to view and contribute descriptions even while away from their computers.

We first implemented a video player module, which would enable visually impaired users to listen to videos while they are away from the computer. Following this, we implemented the authoring mode, which allowed sighted users to record and upload audio descriptions for YouTube videos on an iOS device. Thus, the YouDescribe iOS app recreates the web experience for video and audio playback, recording, searching, sharing and requesting for audio descriptions on social media with full support for low-vision accessibility. The YouDescribe iOS app will not only enable visually challenged users to

enjoy videos on their iPhones, but also encourage volunteers to contribute more in making video content accessible to everyone.

1.2. Existing system

The existing YouDescribe platform consists of the following components:

1) *YouDescribe web client*

The YouDescribe web client has been the primary means to access video and audio clip data for playback and recording. This client is built using HTML5^[13], Javascript^[14] and Bootstrap^[15]. It used the flash-based JWPlayer^[16] as the primary web player for online videos. However, as it used Flash^[54] which is no longer supported by both Android^[17] and iOS, the player and recorder were unavailable on mobile phones. The YouDescribe web client allows sighted video viewers to verbally describe what they see in a movie, and enables viewers to seamlessly share those description clips and their synchronization information over the Internet, thus increasing accessibility of video for blind and visually-impaired viewers.

2) *YouDescribe server*

The YouDescribe server built in PHP^[18] acts as a centralized server to enable communication between the YouDescribe client and the Descriptive Video Exchange (DVX) server. It handles all requests/responses and data transformations between the YouDescribe clients and the DVX server database.

3) DVX (Descriptive Video Exchange)

DVX is a legacy system built by Smith-Kettlewell Video Research and Eye Center. The DVX platform was initially built as a client-server model platform that stores all the description clips, the synchronization information, and the unique identifiers for the original video content on a dedicated database server. Now that the DVX client has been branded as YouDescribe, the DVX server continues to serve this data to the YouDescribe server to make it available to the client. The DVX server can be considered a web service that is built using Java^[19], Hibernate^[20], Spring^[21], JDBC^[22] and MySQL^[23]. It uses an XML^[24] based protocol for accessing the web services.

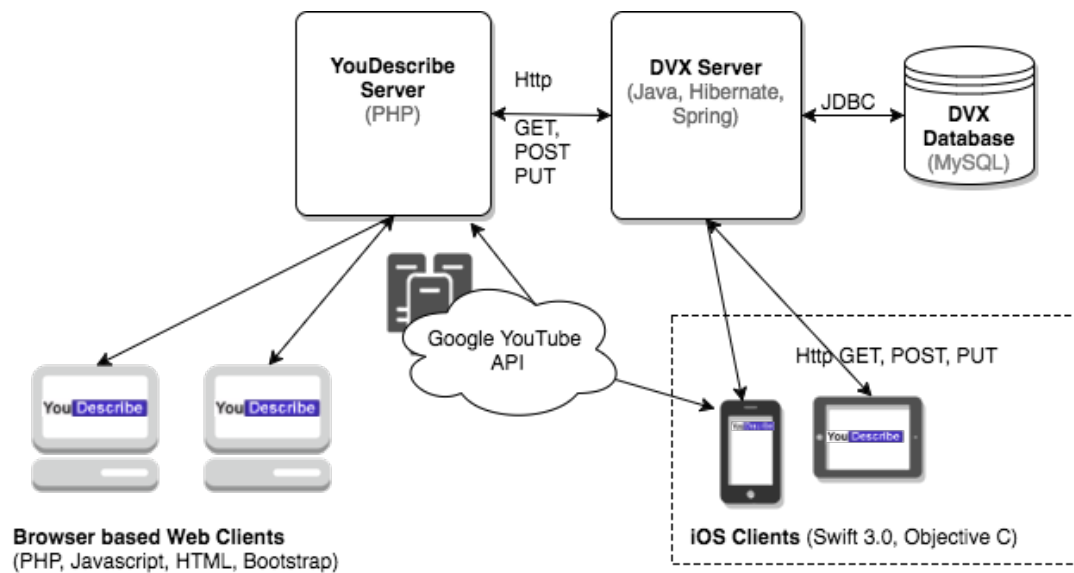


Figure 1: The existing system, along with the proposed iOS client shown in the dotted rectangle

1.3. Roadmap

In this report, we focus on the design and development of the YouDescribe iOS application. We started with introducing ‘YouDescribe’, the existing platform and the objectives of this thesis. We perform a requirements analysis for implementation of the app in Chapter 2. In Chapter 3, we introduce the process adopted in creating a design for the application, giving the reader a view of the initial UI mockups. In Chapter 4, we dive into the implementation details, code structure and accessibility considerations. We also describe the notable challenges faced during the implementation of the project and some solutions adopted to overcome them. In Chapter 5, we then introduce the methods performed to test the application with regards to usability, quality and accessibility. Finally, we conclude by outlining the deployment strategy and future directions.

Chapter 2

Requirements Analysis

The purpose of performing a requirements analysis is to transform high level requirements of the application into well-established, measureable and testable requirements that have been approved by its stakeholders. We first describe the common use cases of YouDescribe iOS app, from which we derive its expected functional and non-functional requirements. We also describe the scope of this project based on various constraints.

2.1. Use Cases

Use cases are developed for an application to define the typical nature of interaction between the user (actor) and the final software product. It outlines the behavior of a

software system from the user's point of view. Given below are some common use cases for the YouDescribe iOS app.

a) Playback of described videos.

- i) Bob, a visually impaired student, is traveling home from school on a bus and is connected to the internet via the data plan on his iPhone. He is used to navigating the iPhone using VoiceOver. He opens the YouDescribe iOS app on his phone, and finds a list of most recently described YouTube videos. He clicks on one of them to open and play them in the player. He hears the audio descriptions being played back at specific points in time during the video playback. Bob is able to understand the visual contents of the video much better through these audio descriptions.
- ii) After launching the video in the YouDescribe player, Bob wonders if any other describers have described the same video apart from the one played by default. He can select his preferred describer 'pixiediesel' from the given list. The same video then plays with the audio description clips of pixiediesel.
- iii) While listening to a video with an audio description, Bob feels that the description volume is too soft. He can adjust the volume of the audio clips.
- iv) While listening to a video, Bob wants to skip over a certain part of the video as it is too long. He can select the given point of time on the timeline at which he would like videos to continue playback.

b) Recording descriptions for videos

- i) Alice is a sighted volunteer; she wants to describe the latest trailer of an upcoming movie to make it available on YouDescribe. She is on vacation and does not have access to her computer. She searches for the name of the movie on the YouDescribe iOS app, and notices that it is not described by anyone else. On logging into her account, she records her audio descriptions and saves them. She previews her audio descriptions in the player to verify if they are acceptable. She can change the start position of the audio clip aligned with the video if she wishes to.
- c) *Searching for videos*
 - i) Trent is a visually impaired user, who is interested in listening to all the audio descriptions recorded by his sister Eva. He is easily able to list all the videos described by Eva, knowing her username.
- d) *Requesting audio descriptions of videos*
 - i) Jay wants to help describe a video but is confused about the video to pick. He accesses the latest Twitter feed for #YDRequest on which he notices several requests for description posted, along with URLs to the description page. He clicks on one of them to directly navigate to the interface that will help describe this video.
 - ii) Eve is a visually impaired user who is interested in understanding the latest innovations in the field of assistive technology. On searching on the YouDescribe iOS app, she finds a related video from YouTube that is currently not described.

She requests a description for the video which automatically posts a description request to Twitter with the hashtag #YDRequest.

2.2. Terminology

This section describes the common terminology used in the project report that shall be kept consistent throughout.

1. DVX: Descriptive Video Exchange, the platform that provides a service to create and retrieve metadata about audio descriptions for existing videos. (See Section 1.4 for more information)
2. Online videos: Videos available over the internet through various websites such as YouTube, Hulu, Vimeo, etc. In the current state of YouDescribe and DVX, YouTube videos are the only supported source of online videos.
3. User: Either a sighted or visually impaired user using YouDescribe who may not be an authenticated YouDescribe user.
4. Descriptor: A sighted volunteer (authenticated through logging in) who records one or more audio descriptions over a video.
5. Described videos: Online videos for which audio descriptions have been recorded by a YouDescribe descriptor.

6. Non-described videos: Online videos for which there are no recorded audio clips by a YouDescribe describer.
7. Audio descriptions: A set of audio clips played back along with the video to help the visually impaired understand the visual content of a video. In the case of YouDescribe, they are recorded by volunteers over online videos, which have synchronization and start time metadata stored in DVX.
8. Inline audio description: An audio clip that is played over the video while the video is playing. Use of inline audio descriptions does not increase the total playback time.
9. Extended audio description: An audio clip that is played over the video while the video is paused. Use of extended audio descriptions effectively increases the total playback time.
10. Player: The YouDescribe player that can play video content along with the recorded audio descriptions.
11. Authoring: A describer recording an audio description for a video.

2.3. Functional Requirements

Functional requirements define the specific behavior or functions of the proposed system. Given below are the functional requirements for the YouDescribe iOS app:

- 1) *Player component – Video and audio playback*

- a) The user (sighted or visually impaired) shall be able to play the listed YouTube video along with its associated inline and extended audio descriptions at appropriate designated start times.
 - b) The user shall be able to skip over sections of the video in either direction (forward or backward) over the timeline
 - c) The user shall be able to choose the describer from among the available describers for the video s/he is interested in. The default describer will currently be the author associated with the video listing. (More details on describer selection are given in Section 2.3.5)
 - d) The video player selected for the app shall be able intercept playback events at a very high rate (accuracy) to enable the audio clip to played at the intended time during video playback.
- 2) *Authoring component – Recording audio clips to YouTube videos.*
- a) The authoring mode shall only be visible to an authenticated user.
 - b) The recorder shall allow an authenticated user to record audio descriptions and upload them to the server.
 - c) It shall allow the user to preview their audio clips over the YouTube video in a player prior to uploading the clips.
 - d) It shall allow the describer to edit properties and delete one or more audio clips belonging to the describer.
- 3) *Login and Registration*

- a) The user shall be able to register for a new account and log in to the website.
 - b) Google authentication shall be supported.
 - c) Without logging in, the user shall be able to browse the website and perform all functions except for recording audio descriptions.
- 4) *Sharing of videos and requesting audio descriptions*
- a) The user shall be able to raise a request for an audio description for either a described or non-described video on social media using the #YDRequest hashtag.
 - b) The user shall be able to share a described video on social media with an aim to increase viewership of the audio described video.
- 5) *Searching for videos*
- a) The home page shall display a list of described videos that are ordered by date of creation.
 - b) The user shall be able to search for videos by video name and/or describer name.
 - c) A user search for a keyword should display a list of described videos matching that query. This list shall be followed by a list of YouTube videos matching that query.
 - d) The user shall be able to view a list of all videos described by a describer.
- 6) *Extended functions*
- a) Feedback system: The user shall be able to publish certain pre-defined feedback to the describer of a video (such as “Needs more description”, “Use more inline descriptions”, “Excellent descriptions!”). This would not only serve to

communicate useful feedback to the describer but also serve to provide motivation to describers to continue describing more videos. The system shall also be able to evaluate the density of audio descriptions over the duration of the video to give pre-defined encouraging and positive feedback to the describer for their work.

- b) Description request system: In addition to allowing users to post description requests on social media, YouDescribe will have its own built-in description request system that tracks YouTube videos and the number of times the video has been voted for. This system shall list these requests in the order of votes. Thus, a potential describer who accesses these description requests could easily gauge the video that is most in demand for audio descriptions.
- c) Selection of the preferred describer: In addition to being able to select the preferred describer while playing back the described video, the user shall be able to set a default preference of the order in which the describers for a video are listed. Some preferences shall include ‘most recent audio descriptions’, ‘longest duration of audio descriptions’ or ‘user’s describer preference’.
- d) Porting metadata from one video to another: Sometimes YouTube blocks a certain video due to copyright violations, or the video’s author takes the video off the public listing. In such cases, if an identical video is available online, the describer shall be able to port existing audio descriptions to another video, so that the effort put into the audio descriptions is not in vain.

- e) Timeline-based visualization and manipulation of audio clips: The authoring component shall include an interface with a timeline representation of both audio and video, that shall allow the user to slide their recorded audio clips over the desired position on the video timeline.
- f) Google authentication: The app shall support authentication of the user via their Google account.
- g) User profiles: Every user shall have a public profile with related YouDescribe description statistics. Users may include links in their social profiles.

All the above described features shall be built with accessibility in mind, particularly accessible to a visually impaired person. They shall be tested for accessibility with iOS's built-in screen reader 'VoiceOver'. The experience while using VoiceOver to navigate the interface through gestures shall be intuitive and shall follow the commonly accepted standards for vision accessibility^[60].

2.3.1. Scope

Due to limitations in the functionality of the current DVX server API and time constraints, we limit our scope to the features supported by the existing YouDescribe website. Thus, through this project we aim to satisfying functional requirements (1) to (5). The extended functions listed in (6) are being considered while building the new

YouDescribe API (version 2). Plans are to subsequently include this functionality in the YouDescribe iOS app. More details are outlined in the ‘Future Plans’ section.

2.4. Non-Functional Requirements

Non-functional requirements are requirements that are implicit while developing the application. They describe the operating environment of the application rather than its behavior. They can be listed as follows.

- 1) The application shall be developed in XCode^[25] on OSX^[26] using Swift 3.0.
- 2) The application shall be made accessible according to well-known iOS standards and guidelines for human interface and accessibility^[27].
- 3) The application shall be optimized for display on all known screen sizes of the iPhone and iPad.
- 4) The application shall be made available to the key stakeholders for periodic testing through TestFlight^[28].
- 5) The application shall be easy and intuitive to use and should require minimum prior training.
- 6) The application shall follow security standards.

2.5. UI Mockups

User interface (UI) mockups are often created in the planning phase for stakeholders to converge upon the general structure of the application's UI. They help prototype the structure and the user's interaction with the final product.

Mockups were created during the planning phase of this app, and were built using fluidui.com ^[29]. Figure 2 shows the launch screen that is displayed while the app is loading. Figure 3 shows the recent audio described videos ordered by time, along with a search bar. The user shall be able to enter part of the title or a describer's name or even search by YouTube ID. The app should also allow the user to navigate to the account or login page. As indicated by the 'Load more' button, a small number of videos (approximately 30) shall be loaded into the table at a time. The home screen should provide an interface to main actions such as login/register and support.

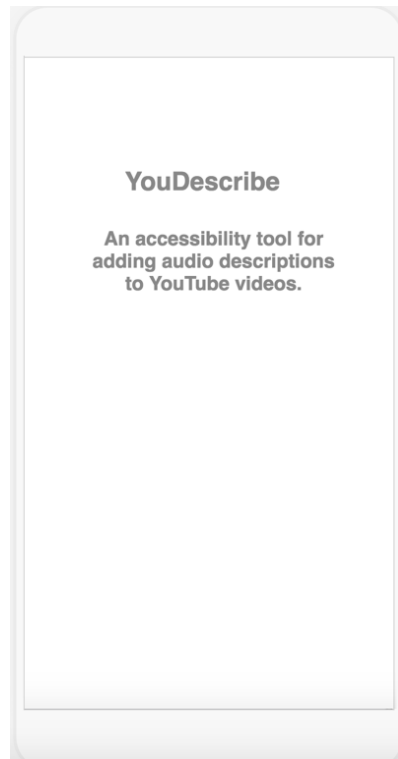


Figure 2: Mockup: Splash screen displayed while the app is loading

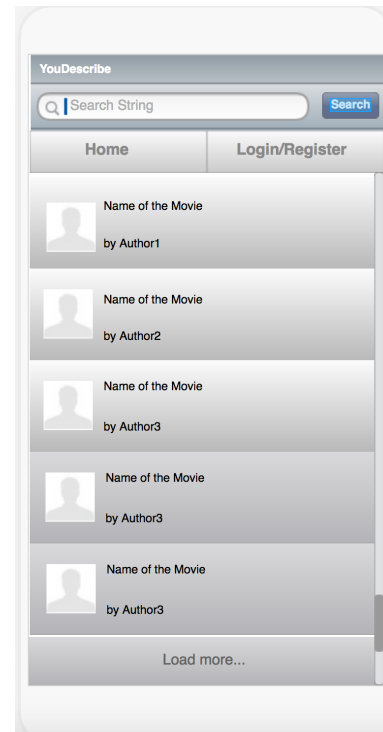


Figure 3: Mockup: Listing of most recent videos on the home screen, with access to the register/login screen. It shall also contain a search bar.

The search results displayed in Figure 4 should allow the user to filter described videos from the non-described ones. The player component has controls for play, stop, skip back, skip forward, In addition, it should provide the ability to share the video.

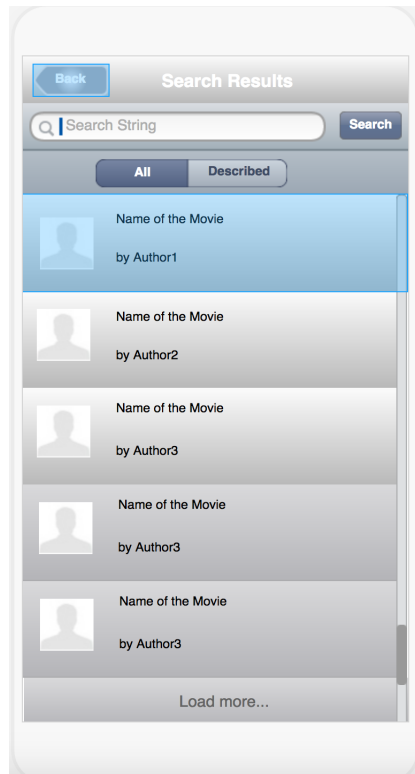


Figure 4: Mockup: Search, and search categories

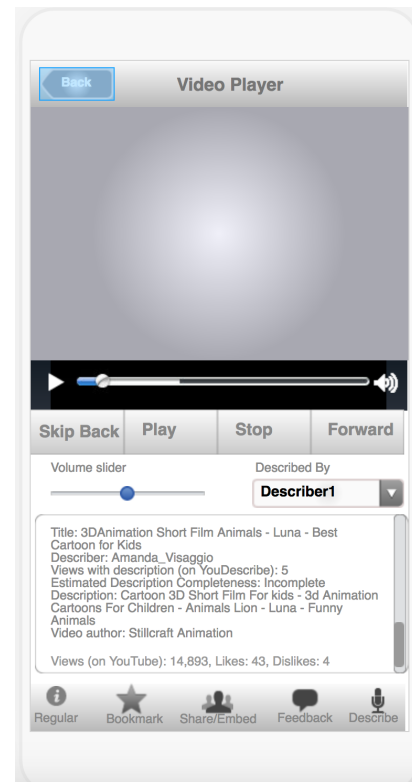


Figure 5: Mockup: YouDescribe player

The design of the player was reviewed by the stakeholders during the course of development and suggestions were made to make the interface more convenient and intuitive to the visually impaired. For example, controls should be made available at the bottom of the screen, and one should be able to request descriptions on social media as well.

Figure 6 shows the mockup for the authoring mode to record audio clips. Here, the describer can play the video with audio descriptions, record a new audio description, or review it before saving. It includes controls to edit the start time and type of the clip and

to delete a previously recorded audio clip. We followed a user-centric design process where feedback was taken from one of the most experienced YouDescribe describers, Dr. Yue-Ting Siu. Dr. Siu suggested that authoring settings and statistics should be moved to the app level, and that the authoring mode be available in portrait mode on the iPhone. Figures 6 and 7 show mockups for the authoring interface and settings. Users can record their audio descriptions through this interface.

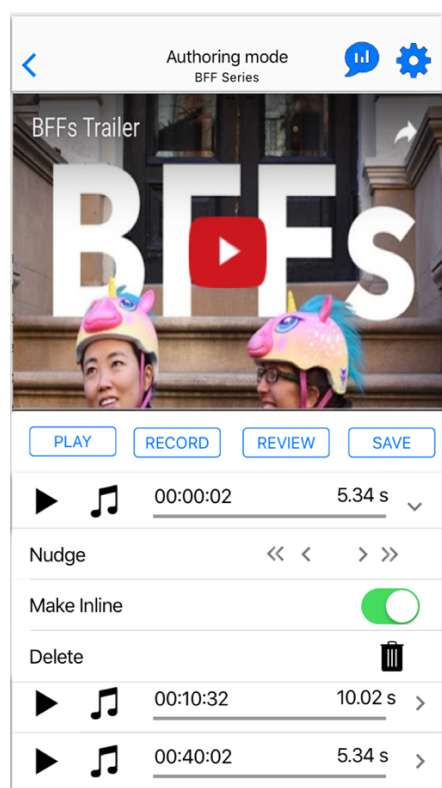


Figure 6: Mockup: The authoring interface that allows users to record audio descriptions over YouTube videos and edit or delete them.

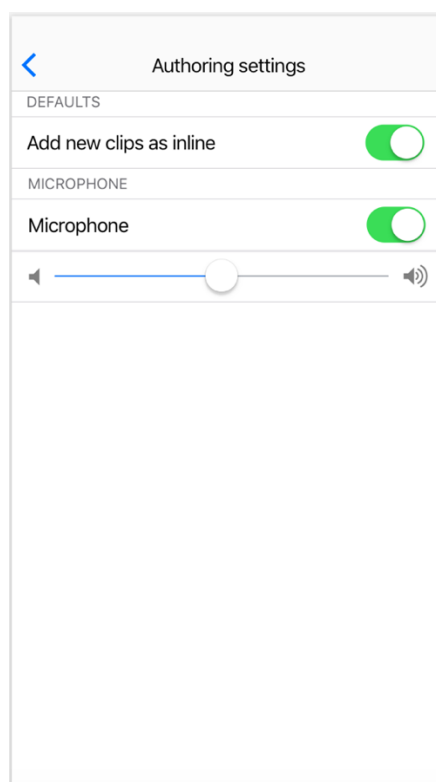


Figure 7: Mockup: Settings for the authoring mode (Recorder)

Chapter 3

System Architecture and Design

This section describes the high level operating and development environments for the YouDescribe iOS app. It outlines the results of evaluating various technologies we used and the decisions behind the selection of certain methodologies to design and develop the app.

3.1. High-level system architecture

This section introduces the technology stack used in this project including the development environment.

3.1.1. Development technologies and Operating system

The project is targeted to support all iOS devices running iOS 9 or greater. The primary language for development is Swift 3.0 with Objective C used whenever necessary.

iOS

The iOS ^[11] operating system (known previously as the iPhone OS) is a proprietary operating system for Apple's mobile devices, namely the iPhone, iPad and the iPod touch. The iOS user interface relies on direct manipulation using multi-touch gestures. iOS reuses some libraries/frameworks with Apple's desktop operating system MacOS ^[26] but the UI toolkit is Cocoa Touch versus Cocoa for the MacOS, and this prevents developers from building apps that run on both. Apps for the iOS operating system must be written in Swift or Objective C and must be compiled specifically for iOS and the 64-bit or 32-bit ARM ^[30] architecture.

Swift 3.0

Swift 3.0 is the third version of Apple's popular Swift programming language used to build apps for iOS, macOS, watchOS, tvOS and Linux. Swift ^[12] is a multi-paradigm compiled language designed to work with Apple's UI frameworks and Objective-C. Swift is considered more robust against programmer errors than Objective-C, and more concise. Swift uses the Objective-C runtime that allows C, Objective-C and Swift code to

execute in the program. Swift has been included with Apple's popular Xcode tool chain since Xcode version 6.0.

3.1.2. Tools and IDE

XCode

The project uses XCode 8 as the primary IDE for development. Xcode ^[25] is Apple's popular integrated development environment (IDE) for building apps on iOS, macOS, watchOS and tvOS. Xcode contains a set of software development tools to craft applications for the Apple ecosystem. Xcode supports development in C, C++, Objective-C, Swift, Java, Applescript among others. Xcode 8 is the latest version of this IDE released on June 13, 2016. XCode also bundles other useful tools for development such as the Simulator, Accessibility Inspector and performance testing tools that aid with the development process.

Version control

Git ^[31] is the primary version control system used for this project. It is a version control system used to track changes to files being worked on by multiple authors/developers. Git is a distributed version control system – which unlike client server version control, every git folder has full-fledged version tracking capabilities without needing network access.

The project is tracked and maintained in GitHub at <https://github.com/SFSU-YDiOS/YouDescribe-iOS>

iOS framework

Cocoa Touch ^[32] is a UI framework for building software programs to run on iOS. Cocoa Touch allows the use of hardware and features that are not found in MacOS computers and are thus unique to the iOS range of devices. Cocoa Touch follows a Model-View-Controller (MVC) software architecture. It is made up of two core frameworks – Foundation and UIKit. Foundation contains the core functionality such as managing strings, collections, etc, while the UIKit contains classes and methods to build user interfaces in iOS.

AVFoundation

AVFoundation ^[33] is an iOS Framework that provides the components to work with audio and video files in iOS.

CocoaPods

CocoaPods ^[34] is a dependency manager for Swift projects that run on the Objective C runtime. It helps resolves dependencies while installing library components for iOS apps known as ‘Pods’.

3.2. Application architecture

For app development using Swift 3.0, the MVC (Model View Controller) architecture has been adopted. In short, the individual terms can be defined as follows:

Model

The model holds the data associated with the system and the logic to process that data. With relation to iOS, model objects (eg. Movie, AudioClip) and parsers live here.

View

The view represents an object (interface) that the user can see, interact with and respond to user's actions. It is usually a reusable component that can be customized differently for presentation. (eg. UILabel is view that presents text on the screen and is reusable)

Controller

The controller manages the communication between the view and the model. It acts as an intermediary between them and usually communicates between them via a protocol. (e.g. A UITableViewController enables the UITableView to communicate with its data source via the UITableViewDataSource protocol). Figure 8 shows that an action performed on the user interface (view) triggers associated logic within the controller. If required, this would update the data contained in the datasource, which would notify with the controller once done. The controller would then update the view with the new data.

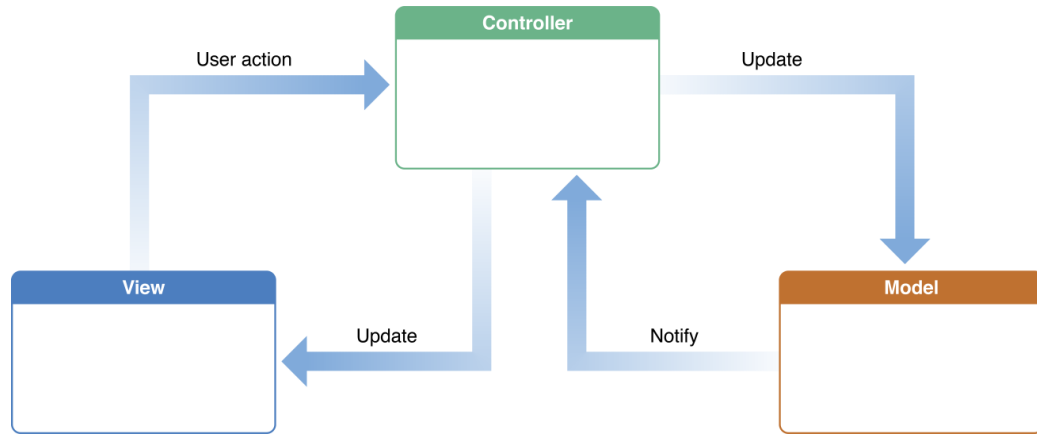


Figure 8: The MVC Architecture (Picture courtesy: Apple developer documentation)

3.3 Evaluation of Third party libraries

3.3.1. Video players

The player component of the iOS app requires that the audio descriptions be overlaid over the original online video. Thus, a video player capable of playing online videos had to be included into the app. Initially, multiple video players were evaluated to play remote content from a URL. Although YouTube videos were to be the primary source of online videos for YouDescribe, we would prefer a player that played other online videos too. To allow fine control over the start time of the audio clip during the video playback, it should be possible to perform events at millisecond playback accuracy. The video player should also have programmatic control over actions such as start, pause, stop, seek. While evaluating the existing players, we soon realized that our choice of players would be restricted by the terms of service of YouTube and the YouTube API ^[55].

Following were YouTube’s notable limiting conditions:

- *“You agree not to access Content through any technology or means other than the video playback pages of the Service itself, the Embeddable Player, or other explicitly authorized means YouTube may designate.*
- *Your API Client will not, and You will not encourage or create functionality for Your users or other third parties to:*
 - *separate, isolate, or modify the audio or video components of any YouTube audiovisual content made available through the YouTube API;*
 - *access any portion of any YouTube audiovisual content by any means other than use of a YouTube player or other video player expressly authorized by YouTube;”*

Next, we describe each of the players evaluated along with their pros and cons.

1) **YouTube iOS Player Helper** ^[35]

This is the official YouTube iOS helper library from Google. It wraps the web-based YouTube iframe player, exposing delegate functions in Swift to respond to common playback events. It is written in Objective C and HTML. It is sufficiently event driven, allowing programmatic start, stop, pause and seeking to a position in time in the video. It adheres to YouTube policies and is currently the standard control used for playing YouTube videos on iOS. Its accessibility features are inherited from YouTube. In addition, it is open-source which would provide low-level control over

its behavior. Although it is an attractive choice, it had a few drawbacks. The time between which events occur while the video is playing is 0.5 second by default. This number is too high to be acceptable, considering that a slight delay in playing back the audio description could adversely affect the listening experience of the user. Additionally, there seemed to be no control for muting the player or adjusting the volume. Also, it is only possible to use one instance of the player on the page at a time.

2) **AVPlayer**^[36]

The AVPlayer is the default controller object by Swift's AVFoundation to play a media asset. It can play local and remote files (via their URL). It can play both audio and video files, with multiple instances on each page. However, it does not contain event handling support by default. It cannot play YouTube videos via its media ID and does not adhere to YouTube policies for video playback.

3) **Swift-YouTube-Player**^[37]

The Swift-YouTube-Player is an open source iOS helper library that makes the YouTube embeddable iframe available in iOS applications, like Google's YouTube iOS Helper player. It adheres to YouTube policies and contains controls to start, stop, pause, and seek to a point in time in the video. It allows convenient installation through Pod installation^[34]. It is easy to extend as the API is written in Swift and can

be compiled to add more functionality. However, it does not seem to expose an event that is invoked during playback, which makes it impossible to perform audio playback at certain points during the video. Like the official player, there is no support for volume control exposed by the API.

4) **JWPlayer**^[16]:

JWPlayer is a commercial video player by LongTail solutions^[38] that can be used for development. It includes support for the web, Android and iOS platforms. At the time of performing this evaluation, its flash-based web player was used by the YouDescribe web client. Its playback could be intercepted at every 0.1 seconds which seemed to be ideal for this use case. It would allow audio clips to be played by at accurate start times during the video, thus providing the desirable user experience. It is event driven, with control over start, stop, pause, and seek as well. It includes an analytics platform that could provide useful statistics and integrate with the web-client. However, at the time of player evaluation, JWPlayer did not provide native support of YouTube in their iOS control, but it was only enabled through the web-based HTML player^[39]. As it is a commercial component, it is not free for use and purchasing a license would result an additional cost.

Table 1 is a comparison chart that summarizes the key considerations.

Table 1: Comparison of iOS video players

Parameters	YouTube iOS Player Helper	AVPlayer	Swift-YouTube-Player	JWPlayer
Description	Official Google control for playing YouTube content in iOS	iOS native video/audio Control	Open source implementation of YouTube iframe API for iOS	Commercial video player
Open source	Yes	No	Yes	No
All video types	No	Yes except YouTube	No	Yes
Understands YouTubeId	Yes	No	Yes	Yes (only iframe)
Adheres to youtube policies	Yes	No	Yes	Yes
Start/Stop/Pause	Yes	Yes	Yes	Yes
Events	Yes	Few	Yes (fewer)	Yes
Accessibility support	Yes	Yes	Yes	Yes
Resolution of playback events	Approx. 0.5 seconds by default	Not available	Not available	Approx. 0.1 seconds

It was concluded that Google's Open source YouTube help player control (1) should be used for this app, as the current system focuses on describing YouTube videos only. However, the player interface should be built with an interface such that the player can be swapped out if required. We could overcome the limitation of playback event accuracy by modifying the open source player to use 0.1 second instead, which produced the desired user experience with no noticeable artifacts.

3.3.2. Audio players

1. AVPlayer

Belonging to the AVFoundation framework, the AVPlayer is a generic media player for video or audio playback from a local or remote file. It can control the playback and the timing of a media asset. It can perform programmatic play, pause, change of playback rate and seek to points within the playback timeline. It can also be used for HTTP based live streaming. It allows the use of the same AVPlayer instance with different AVPlayerItem's attached to it.

2. AVAudioPlayer^[40]

Belonging to the AVFoundation framework, the AVAudioPlayer allows playback of an audio file either from a local file on disk or from a memory buffer. One instance of the AVAudioPlayer can only be associated with one file at a time.

From our evaluation, we concluded that the AVPlayer was suitable for audio description playback. In our app, multiple audio descriptions are to be loaded and played one after the other at specific start times during video playback. As AVPlayer provided the flexibility to play multiple audio files using the same instance of the player it proved to be more suitable. Also, our app might want to stream HTTP audio files in the future; hence we chose AVPlayer as it allows streaming audio files from an HTTP URL.

3.3.3. Alamofire^[41]

Alamofire is a popular, elegant networking library for Swift. It allows basic request, response handling, validation, chaining and automatic parameter encoding. It also supports download of large data files with progress reporting and the ability to resume an interrupted download. Due to the current encoding of clip files as ‘application/zip’ in DVX, download of audio clips to the temporary storage before playing them back was mandatory. Due to this, the Alamofire networking library was chosen.

3.3.4. SwiftyJSON^[42]

As Swift is strict about its types and allows good explicit typing, JSON format is more implicit about its types. Thus type handling with the default JSONSerialization module becomes tedious. The SwiftyJSON library makes this more elegant and parses it into a simple dictionary.

3.3.5. LAME encoder^[43]

iOS does not natively support the creation of MP3 files on the device as it is a proprietary format ^[56], though it fully supports the reading and playback of MP3 files. As the audio files on the DVX server are encoded as MP3 files, it was necessary that the audio files written out by the iOS YouDescribe recorder also retain the same format for consistency. The MP3 Lame encoder was used for this purpose to encode the bytestream from AVAudioEngine as MP3 and write it to a file on disk which is subsequently uploaded to DVX server.

3.4. Application programming interfaces (APIs)

To enable communication with the server, REST ^[44] API endpoints were exposed to the client. Representational State Transfer (REST) is an architectural style that specifies constraints, such as the uniform interface, that if applied to a web service helps with performance, scalability, and modifiability that enable services to work best on the Web. In the REST architectural style, data and functionality are considered resources and are accessed using Uniform Resource Identifiers (URIs), typically links on the Web. The resources are acted upon by using a set of simple, well-defined operations. The REST architectural style is similar to a client/server architecture and is designed to use a stateless communication protocol, typically HTTP. Here clients and servers exchange representations of resources by using a standardized interface and protocol. By

implementing a server with RESTful architecture, it is guaranteed that each HTTP based API give every type of resources in the system its own URI, “uniform interface” (GET, PUT, POST, DELETE) are used on URIs to work with resources, and all request are stateless such that each request is independent of any previous request

3.4.1. The DVX API^[45]

The development of this application requires communication with the DVX (Descriptive Video Exchange) server via the DVX API. This is a REST like API that responds with XML documents of lists of objects containing data from the DVX Database. The structure of the DVX database schema is shown below.

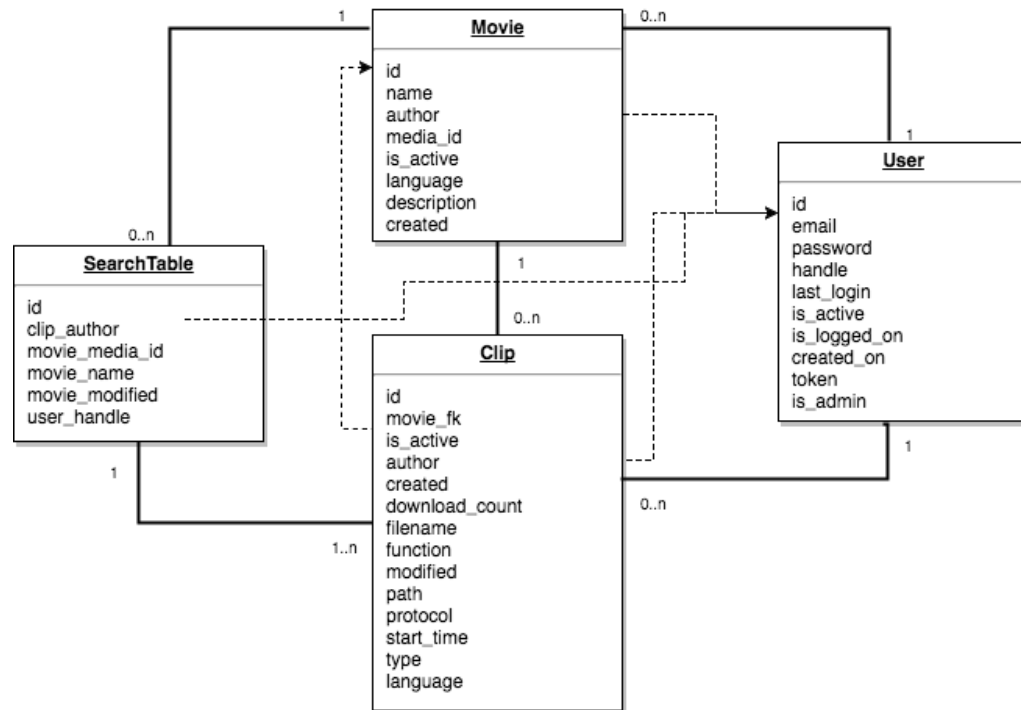


Figure 9: Schema of important tables in the DVX database

The data associated with the above attributes are retrieved and updated through the DVX API. It is accessed through endpoints with the base URL <http://dvx.ski.org:8080/dvx2Api/> . Following are endpoints used for application development

Table 2: Summary of the DVX API

Name	Type	Description
movie	GET	Retrieves metadata for one or more movie
movie	POST	Creates a new movie entry, unique to a single YouTube media ID.
searchTable	GET	Retrieves all the search results ordered by time.
clip/metadata	GET	Retrieves the clip metadata for one or more clips
clip	GET	Downloads the clip file saved on the server
clip/metadata	POST (requires login)	Creates a new clip metadata entry
clip/upload	PUT (requires login)	Uploads a recorded audio clip.
user	GET	Retrieves information about one or more users
user	POST	Creates a new user with the given credentials and sends out a confirmation

		email to the address
login	GET	Authenticates the user's login credentials and returns the session token.

3.4.2. The New YouDescribe API (Version 2)

During the last week of April 2017, a part of the new API that was being built for YouDescribe was exposed to us. Following are the notable differences compared to the existing system. It now uses a new backend technology with NodeJS^[57], MongoDB^[58], Mongoose^[59]. The new API produces JSON formatted responses with nested objects. It includes endpoints to access a description request system known as the 'Wishlist'. The format of the recorded audio clips was changed to WAV from MP3. This new API would replace the need for the current YouDescribe (PhP) and DVX (Java) servers. Table 3 summarizes the currently functional v2 API endpoints with their parameters:

Table 3: Summary of the new YouDescribe REST API

Name	Parameters	Type	Description
videos	page	GET	Retrieves 30 video entries at a time based on the page number. Retrieves all videos without this parameter
	q	GET	Retrieves videos based on the search query
	{YouTube	GET	Retrieves video details about a

	mediaID} eg. B8BD9txkGL4		particular YouTube video
wishlist		GET	Retrieves a list of all videos that have been requested for description based on the number of votes.

Migration to the new YouDescribe API is currently an ongoing task. The document shall primarily only focus only on application development with the DVX API.

3.4.3. Third party API: YouTube Data API^[46]

The YouTube Data API provides REST API endpoints to perform functions related to data on the YouTube website, such as listing search results and retrieving additional information about a video. The lists below identify the different types of resources that you can retrieve using the API. The API also supports methods to insert, update, or delete many of these resources. It requires that the request be made with an API key or an authorization token. Please refer to appendix [3] for details regarding the structure.

3.5. Typical sequence of data flow

To understand the interaction between various systems it is necessary to break down a simple task into a sequence of events that describe the data flow across these systems.

Figure 10 shows the flow of data across different levels to display the most recently

described YouTube videos (movies) and its metadata when the app is launched. The home screen is loaded as soon as the videos are retrieved from the DVX server, while the images and video duration data are loaded asynchronously.

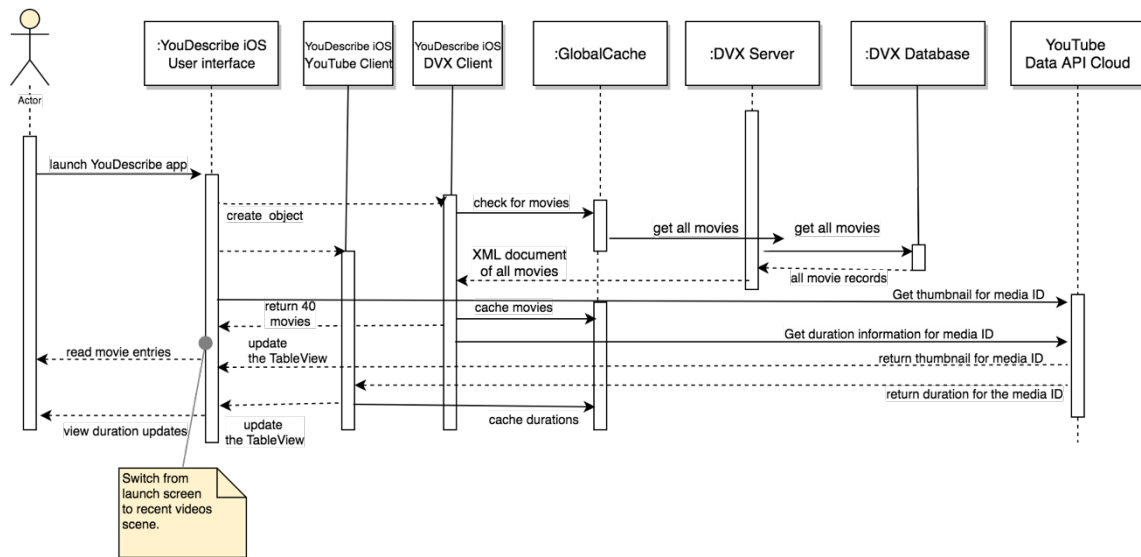


Figure 10: Sequence diagram illustrating the interactions between various systems to display most recent videos to the user

3.6. Accessibility considerations

To develop applications that are accessible to the blind and visually impaired, it is necessary to understand how the visually challenged use mobile devices. Such users perform common functions on their phone with the help of one or more assistive tools such as screen readers. VoiceOver^[47] and TalkBack^[48] are very popular screen reader applications for iOS and Android users respectively. Screen readers are software

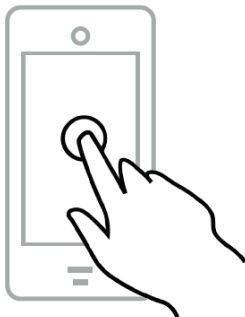
programs that identify the elements displayed on a screen and repeat that information back to the user, via text-to-speech or braille output devices.

As Apple has a long tradition of including accessibility into their software products, the usage of their productions by the visual impaired community is very high. The VoiceOver screen reader has been built into iOS since version 3.0. VoiceOver is a gesture-based screen reader which reads out elements on the screen in order so that people with low vision can understand the interface. Since sequentially scanning the page might tend to be taxing, the visually impaired could also navigate directly to an element on the screen through a long press over the element. Thus, adopting common interface standards to develop apps is necessary. Not only is VoiceOver highly customizable on the fly via the ‘Rotor’ ^[49], but it is also easy to learn. This makes it more popular than other screen readers on iOS based mobile devices. VoiceOver can also be used with a connected Braille keyboard.

Enabling VoiceOver on the iPhone can be done through Settings > General > Accessibility > VoiceOver . The triple click home button can also be enabled as the Accessibility shortcut to start VoiceOver. Given below are some of the VoiceOver gestures used to navigate the iOS device (See Reference 50 for source of images)

Touch Screen

Select and speak the item.

**Double Tap**

*Activate selection
(open app, press button)*

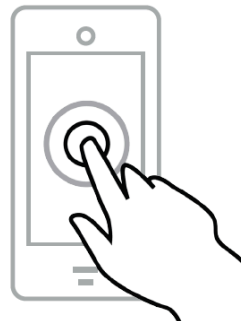


Figure 11: Gestures and their functions in iOS VoiceOver

Double Tap and Hold

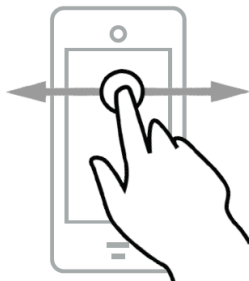
*Activate Drag Mode
(move icons, swipe-delete)*

**Triple Tap**

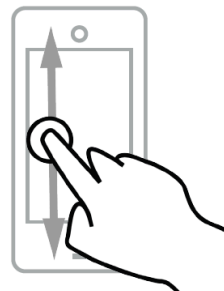
Double press button

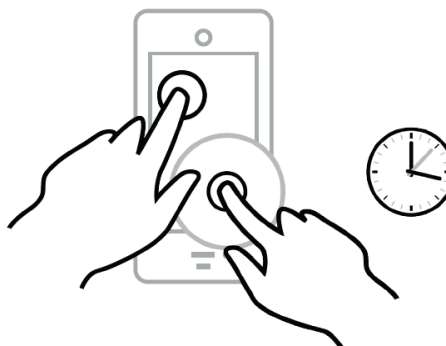
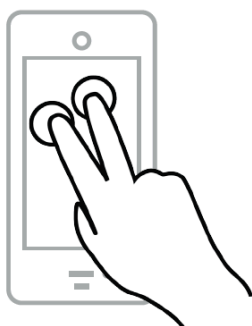
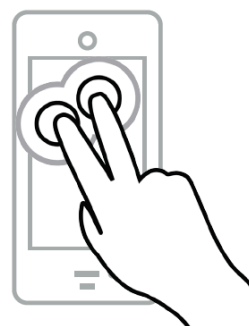
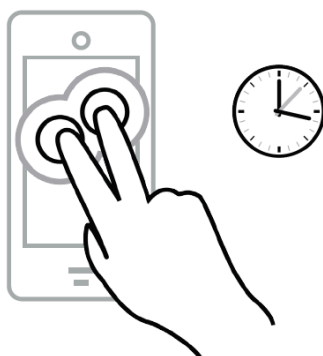
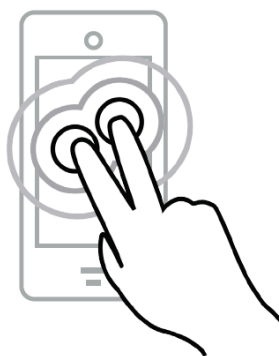
**Swipe Left/Right**

Select previous/next item

**Swipe Up/Down**

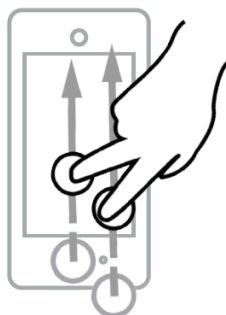
Select previous/next selected rotor option



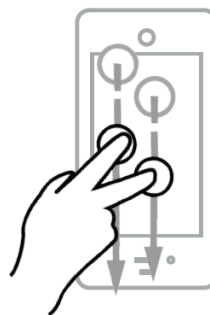
Double Tap Status Bar*Scroll to top***Hold One Finger, Tap Another***Select when dragging, activates with tap***Two Finger Tap***Pause/resume speech***Two Finger Double Tap***Performs special action***Two Finger Double Tap & Hold***Add label to selected item***Two Finger Triple Tap***Open item chooser for current area*

Two Finger Swipe Up

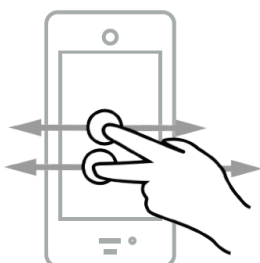
Read all from first object in selected area

**Two Finger Swipe Down**

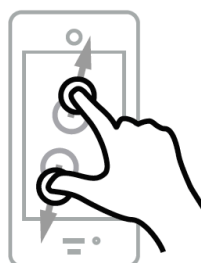
Read all from selected item

**Two Finger Scrub Back & Forth**

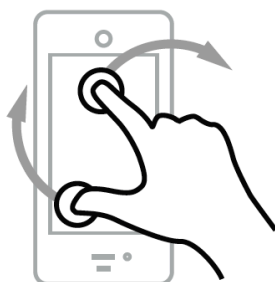
Go back, cancel, close pop-up

**Two Finger Pinch Out & In**

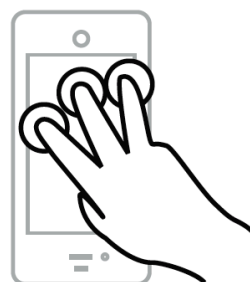
Select and deselect

**Two Finger Rotate**

Select next or previous rotor item

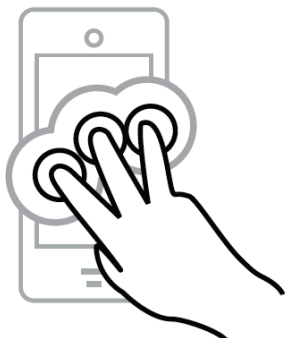
**Three Finger Tap**

Speak the current page number and position



Three Finger Double Tap

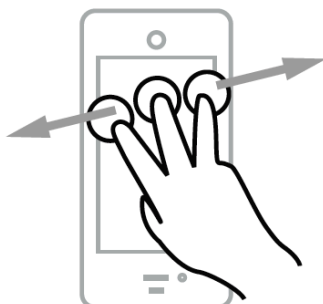
Toggle speech on/off

**Three Finger Triple Tap**

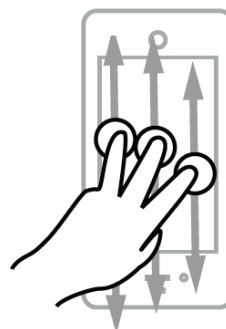
Toggle screen curtain on/off

**Three Finger Swipe Left/Right**

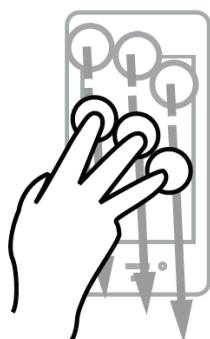
Move to previous/next screen or page

**Three Finger Swipe Up/Down**

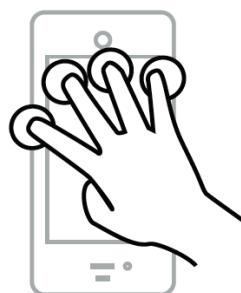
Move or scroll up/down

**Three Finger Swipe Down from Status Bar**

Open notifications center

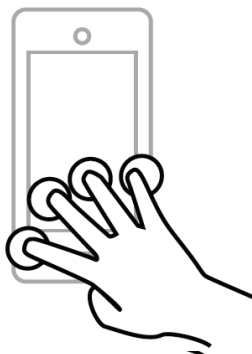
**Four Finger Tap (Upper Half)**

Select first item in the area

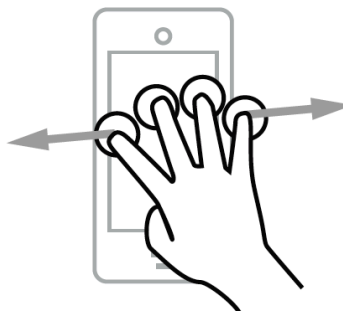


Four Finger Tap (Lower Half)

Select last item in the area

**Four Finger Swipe Left/Right**

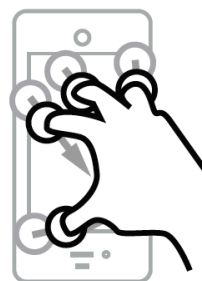
When in an app, switch to previous/next app

**Four Finger Swipe Up/Down**

Open or close multitask pane

**Four Finger Pinch**

Go to home screen



Chapter 4

Implementation

This chapter documents the development tasks, organization of code and details about Swift 3.0-specific elements used throughout the project. It also discusses the development tasks to make the interface accessible.

4.1. Tasks and Milestones

Following is a list of the implementation tasks and milestones based on the project schedule.

- 1) Prototype, evaluate video players, their APIs and suitability.

- 2) Develop a standalone player over the YouTube player that can play inline and extended audio descriptions
- 3) Develop the video listing home screen, along with the search functionality and filters.
- 4) Develop the supporting functions for the player such as describer selection, skip forwards, backwards
- 5) Develop supporting functions for sharing the described video and requesting for a description on social media.
- 6) Develop supporting functions for computing and displaying estimated description completeness, and associated metadata (likes, dislikes, views)
- 7) Develop login and registration with session handling.
- 8) Implement volume control and audio session handling.
- 9) Prototype recording capabilities of MP3 files on iOS.
- 10) Develop the authoring interface, allowing recording, preview and upload of inline and extended audio clips.
- 11) Allow editing and deletion of audio descriptions.
- 12) Embed the YouDescribe player within the authoring interface.
- 13) Embed a timeline view to display the distribution of video and audio clips for a described video.
- 14) Develop app caching and video refresh functionality.

From the above list, (1) to (4) were milestones achieved from September 2016 to January 2017. (5) to (14) were achieved during the period between February 2017 and May 2017.

4.2. Class diagram

UML (Unified Modeling Language) is a language widely used to visualize and capture the behavior of complex software systems and the interaction between its components. In UML, a Class diagram models the static view of the system, including relationships between multiple classes and their functions. Figure 12 presents a class diagram for the iOS app. In the diagram, the classes in yellow are built-in iOS classes. The DvxApi and YouTubeApi modules are represented separately for the purpose of keeping the diagram readable. We can assume that all view controllers obtain an instance of either one or both modules to derive data needed to update the respective view. Note that some connections between classes are represented in blue to avoid confusion that might arise due to overlapping lines.

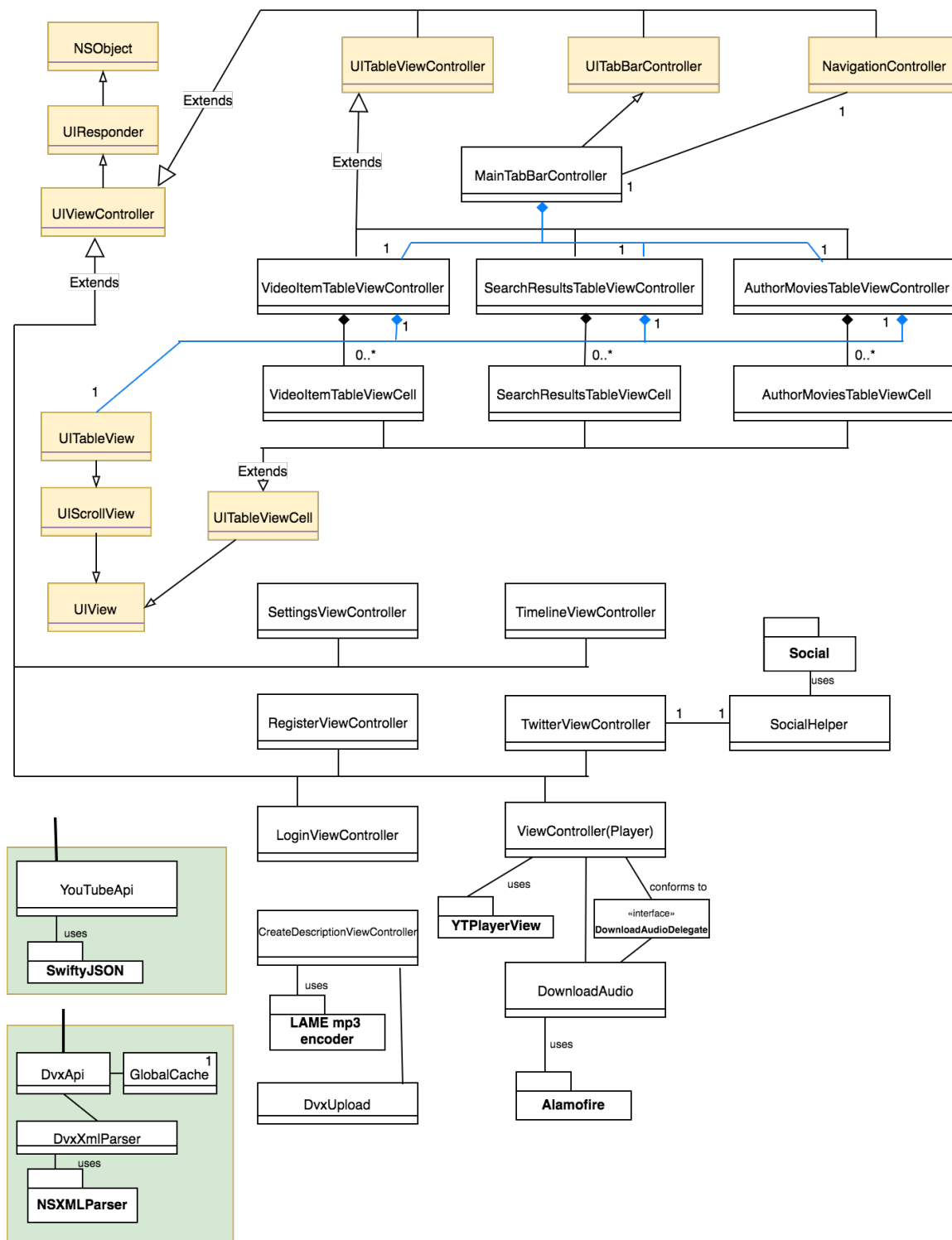


Figure 12: Class diagram of the YouDescribe iOS app

4.3. Code Structure

Given below is the code structure of the YouDescribe-iOS project. At a broad level, the structure is arranged in model, view, controller, helpers, globals, api.

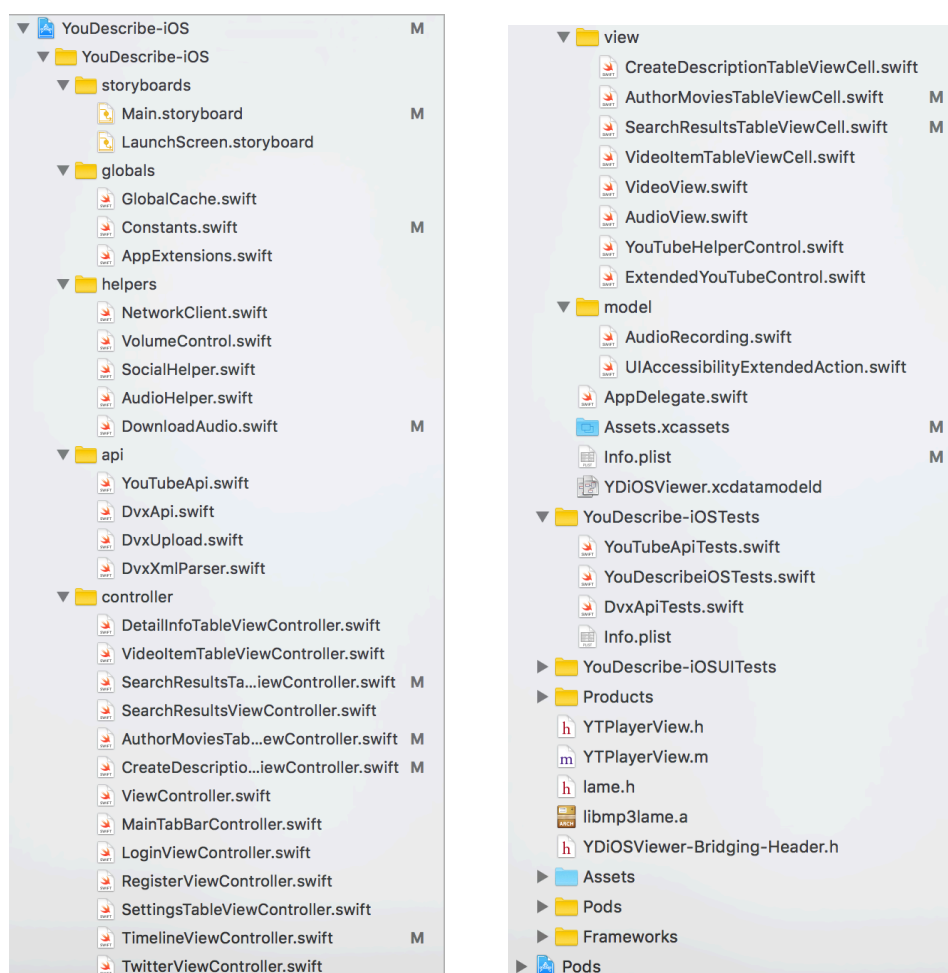


Figure 13: Code Structure

4.3.1. Main Project (YouDescribe-iOS)

This file sits at the top of the hierarchy and is used to provide fine grain control over all aspects of the app deployment, be it bundle identifiers, permissions, or library

dependency frameworks. A unique bundle ID needs to be specified for each app for deployment. For the YouDescribe project, the bundle ID is com.youdescribe.YouDescribeiOS. It also allows the developer to specify the version, build number, team and authentication information related to signing the app. It identifies the target deployment environment which is 9.0 for this app. We modify custom iOS target properties to suit the app's need. They include:

1. Privacy settings for Microphone usage: This setting is required to get access to the device's microphone to allow recording of audio descriptions.
2. App transport security settings to allow arbitrary loads: This setting is required to allow Swift to make calls through HTTP.

4.3.2. AppDelegate.swift

This serves as the main entry point of the app. The @UIApplicationMain attribute creates a run loop that delivers input events to the app. It manages the life cycle of the app and the delegate object. It provides methods to run custom code when the app is launched and at different stages in the app's life cycle. It also provides control over Core data in the app, mapping the managed object models and persistent data to classes.

4.3.3. LaunchScreen.storyboard

To give the feel of a responsive app while the app is loading, the launch screen UI is made to display text or an image.

4.3.4. Info.plist

The information property list file is a list of key-value pairs representing metadata used by the app or bundle for its internal use. It stores configuration information in a place where the system can access it easily. On disk this file is represented as XML.

4.3.5. Assets.xcassets

This directory is the asset catalog that holds image assets for three resolutions (1x, 2x, and 3x). It displays the appropriate image based on the device.

4.3.6. Bridging-Header.h

This is an Objective-C header file that enables the use of Objective-C code within existing Swift code. On importing packages in this file, it includes the file in the Build settings and is built during code compilation. This project enables importing of the YouTube player helper library and liblame mp3 encoder through the bridging header.

```
1  //  
2  // Use this file to import your target's public headers that you would like to expose to Swift.  
3  //  
4  
5  #import "YTPlayerView.h"  
6  #import "lame.h"
```

4.3.7. Globals

The Globals directory groups code that is used by code throughout the app. It contains the Constants which contain app-specific global variables and configuration values. It includes application specific extension functions and cache handling.

1) Extensions

It is possible to add new functionality to an existing class or structure using extensions. Using ‘retroactive modeling’, the developer can also extend types that are built-in with no access to the code. Some custom extensions include those for loading an Image into an UIImageView asynchronously, or dismissing the keyboard on tapping anywhere on the screen, as shown below.

```

12 extension UIViewController {
13     func hideKeyboardWhenTappedAround() {
14         let tap: UITapGestureRecognizer = UITapGestureRecognizer(target: self, action:
15             #selector(UINavigationController.dismissKeyboard))
16         tap.cancelsTouchesInView = false
17         view.addGestureRecognizer(tap)
18     }
19     func dismissKeyboard() {
20         view.endEditing(true)
21     }
22 }
23
24 extension UIImageView {
25     public func imageFromServerURL(urlString: String) {
26
27         URLSession.shared.dataTask(with: NSURL(string: urlString)! as URL,
28             completionHandler: { (data, response, error) -> Void in
29
30             if error != nil {
31                 print(error!)
32                 return
33             }
34             DispatchQueue.main.async(execute: { () -> Void in
35                 let image = UIImage(data: data!)
36                 self.image = image
37             })
38         }).resume()
39     }
40 }

```

2) *NSCache*

Caching is used to store to objects in memory that are expensive to create. *NSCache* is a mutable collection that stores key-value pairs. It also uses smart eviction policies in case too much memory is being used by the cache. It helps to prevent duplication of large number of records in memory within the app thus reducing memory footprint. As the app heavily relies on data stored in DVX, this data is cached and used throughout the app. Adding a new movie during description will invalidate the cache storing movie objects thus causing it to query the database again and cache it.

4.3.8. Helpers

This group holds classes that assist with a particular set of functions within the iOS app, and can be reused throughout the code. They include

1) *SocialHelper*

It uses iOS' Social module to share and request descriptions on available social media applications installed on the user's device.

2) *AudioHelper*

This module provides core functionality of the recorder for the authoring mode. The PCM data read via the *AVAudioEngine* on recording, is read into a buffer of size 4096 and

encoded into MP3. This MP3 encoded buffer is appended to raw file data which is written to disk once the recording is stopped.

3) *DownloadAudio*

This module handles the download of audio description clips into the application cache. It can be configured to handle both asynchronous and synchronous download of audio clips. The app currently uses synchronous sequential download of audio clips, due to some issues observed with the test DVX server. This module also handles tracking the files as they download through a *DownloadAudioDelegate* module.

4.3.9. API

This group contains wrapper files for various API endpoints used by the app. The *DvxXmlParser* is used to parse the XML documents returned by calls to the DVX API endpoints. It implements the *XMLParserDelegate* to override functions to parse the root XML element along with the element name to return an array of freeform objects (*AnyObject* type) . The YouTube API wraps functions such as search information, video details, video duration. *DvxUpload* acts as a helper to the *DvxApi* for uploading multipart audio clip information with MP3 data.

4.3.10. ViewControllers

ViewControllers are the primary components of an app's architecture. They manage the views connected to them and coordinate the flow of data between the app's model and the

views that display data. Customizations to the existing view controllers are performed for video listing, login, registration, player and authoring mode. As shown in Figure 14, different methods are called during different stages in its life cycle.

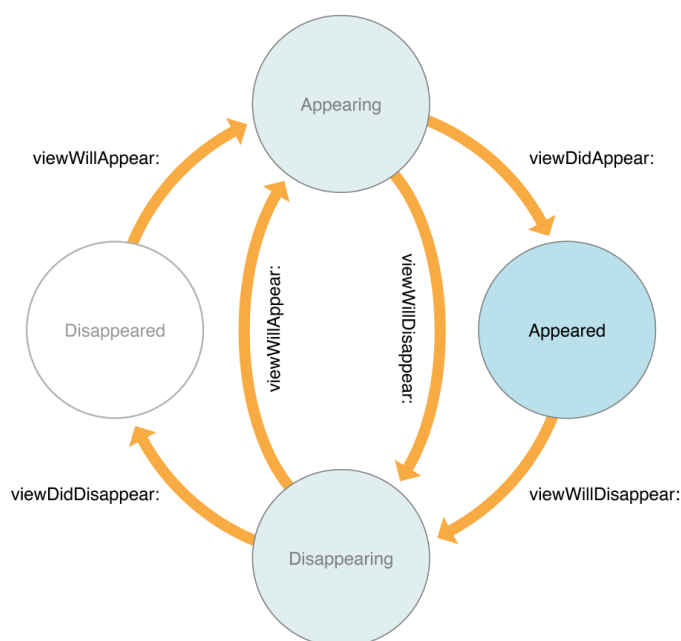


Figure 14: Methods invoked during the life cycle of a view controller (Courtesy: Apple Documentation)

4.3.11. Views

This includes custom UIViews for the prototype cells of the TableView. These views in turn hold other views such as labels, buttons, images and outlets to their event callbacks.

4.3.12. Pods

Cocoapods is a dependency manager for Swift Library components (Pods). We use the Alamofire networking library and SwiftyJson in the project as described in the ‘Third

party libraries' section. Pods need to be installed via a Podfile to make them available for use in the project. Figure 15 shows the Podfile used for the YouDescribe iOS app.

```
1 source 'https://github.com/CocoaPods/Specs.git'
2 platform :ios, '10.0'
3 use_frameworks!
4
5 target 'YouDescribe-iOS' do
6   pod 'Alamofire', '~> 4.0'
7   pod 'SwiftyJSON', '3.0.0'
8 end
9
```

Figure 15: Podfile used by the YouDescribe iOS app

4.4. UI Implementation and functions

The YouDescribe app was designed and built keeping the iOS human interface guidelines^[27] in mind.

4.4.1. Cocoa Touch UI concepts

1) Constraints

Constraints are the relative or absolute spacing (top, bottom, width, height) between elements in the UI, essential to create adaptive layouts suitable for multiple devices.

2) Segues

A segue is a means to transition between view controllers. It is possible to define a relationship between the view controllers such as 'Show', 'Embed', 'Push' etc, based on the hierarchy setup between them. A segue is assigned a unique identifier that used to identify the target view controller and pass data to it. It is commonly used to pass data from the parent to the child view controller.

3) iOS Delegates

Delegation is a callback mechanism commonly used in iOS to enable communication when information needs to be passed from a child object to the parent object. For example, by implementing the `UITableViewDelegate` methods, the base `UITableView` is able to delegate the behavior of certain built in functions to the app.

4) NotificationCenter

The `NSNotificationCenter` is a way to notify one or more classes or instances about an event that occurred. It acts as the central hub to receive communications and most often used so that classes can take appropriate action based on the communication they receive. The observing class registers for a particular notification using a key, and is notified as soon as a message associated with the key is posted to the notification center.

5) Outlets and Actions

Outlets and actions are means to connect the UI Controls to Swift code. `IBAction` (Interface Builder Action) attribute allows an event on a specific UI Control to trigger a piece of code. `IBOutlet` (Interface Builder outlet) attribute provides a reference within the Swift code to access the properties of a UI control.

4.4.2 Interface Builder (IB)

The Interface builder (IB) is an editor within XCode that simplifies UI Design by allowing the creation of complete storyboards containing views, properties, relationships separate from the implementation. The Cocoa Touch user interface objects are archived as .nib files. It provides useful functions such as

- 1) Auto Layout: This feature allows the user to specify UI constraints for each of the controls to make sure they form a set of consistent rules adaptable for different screen layouts.
- 2) Preview: This mode allows the developer to test the layout of the controls on a variety of device layouts in both landscape and portrait orientations, speeding up the iterative design process.
- 3) Storyboard: An iOS app is composed of multiple views that the user navigates. The relationship between these views is defined as a storyboard. It represents the app's flow.

The storyboard for the YouDescribe app is as given in Figure 16.

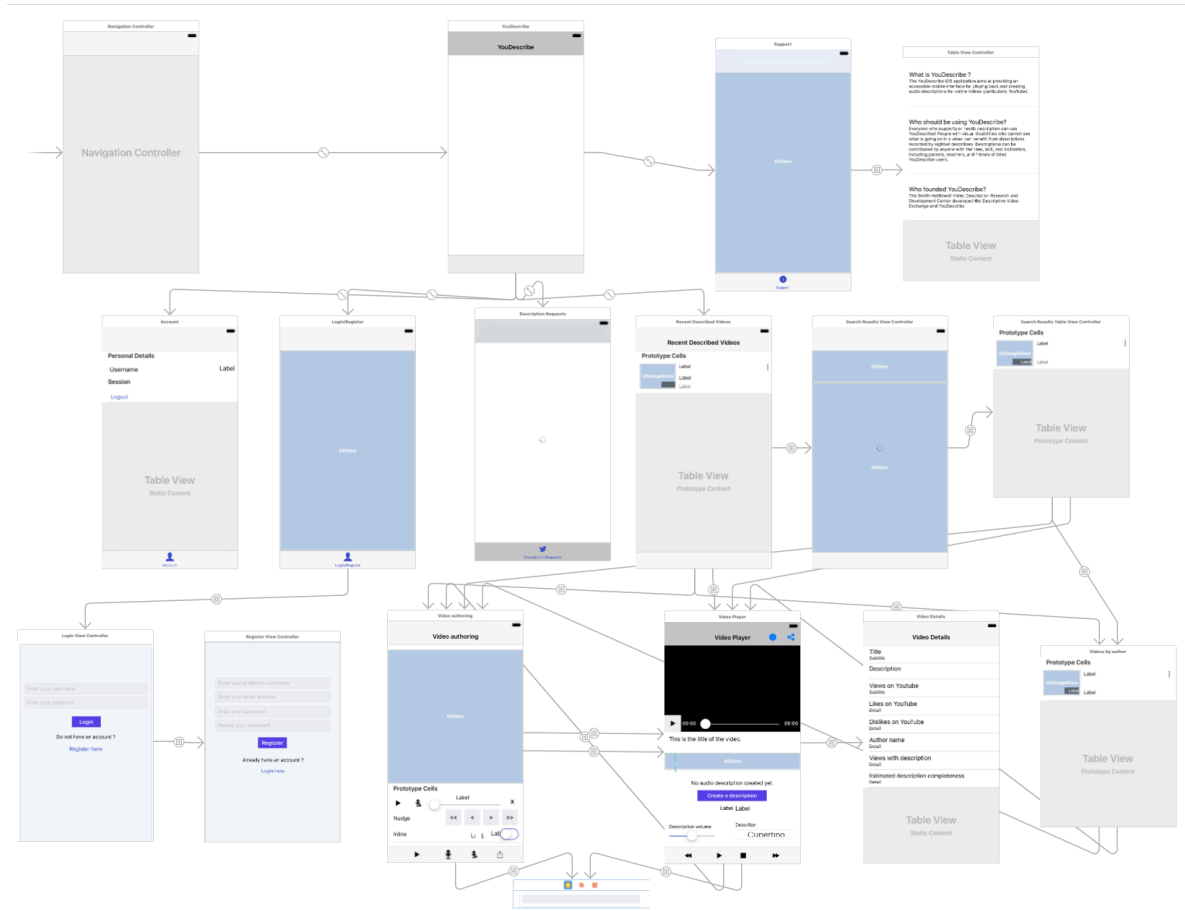


Figure 16: Storyboard for the YouDescribe iOS app. The junctions joining the view controllers represent ‘Segues’

4.4.3. Home screen

The Home screen displays video listings of the most recently added videos with audio descriptions. The Main screen is a **TabBarController** displaying videos, account information, description requests and support. Each element within the table is

configured using a Prototype Cell for the table. It provides additional options to create a new description and list other videos by the describer through the Details menu.

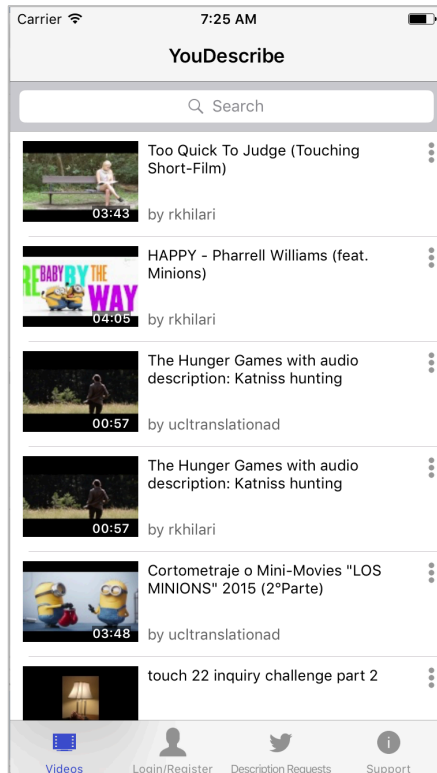


Figure 17: Home screen showing most recent described videos

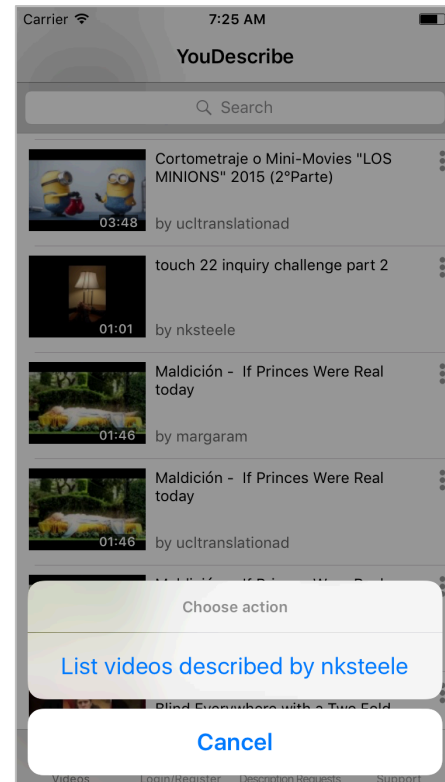


Figure 18: Detail menu options for each video

4.4.4. Managing user sessions

The user can login with their username and password. If they have not already registered, they are prompted to create an account with a username, email address and password. On accessing the confirmation link sent by the DVX API on registering, the user is allowed to login.

Upon being authenticated, the DVX API returns a four-letter token that is saved locally and saved as session information in the ‘UserDefaults’. NSUserDefaults shared defaults object stores user specific, persistent, immutable key-value pairs. Figure 19 shows the login screen while Figure 20 shows the registration screen for a new user.

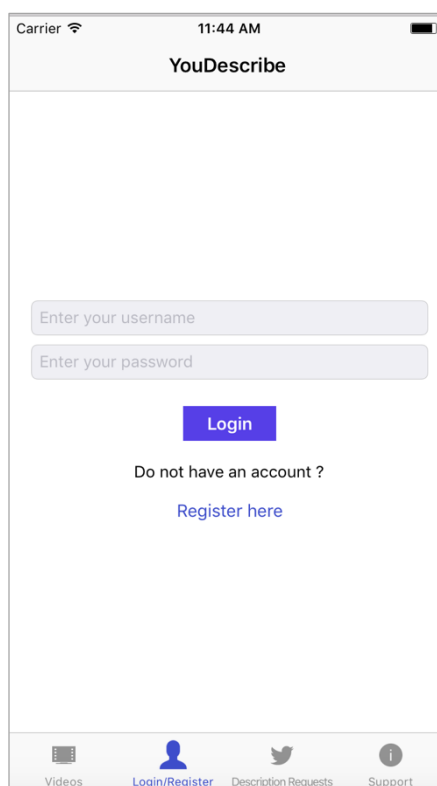


Figure 19: The login screen

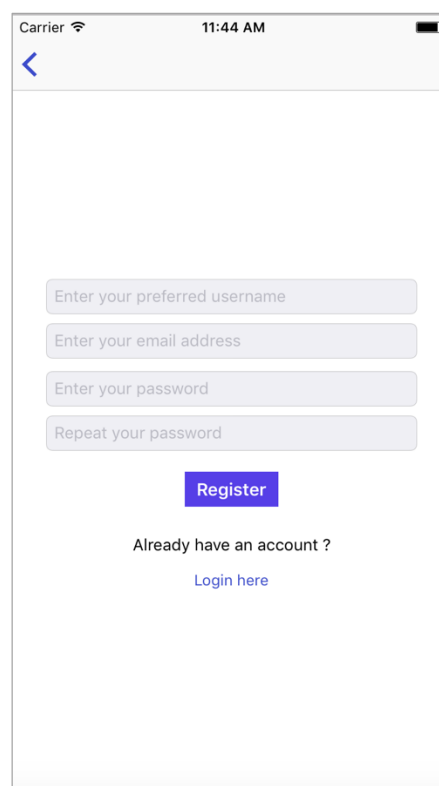


Figure 20: The user registration screen to create a new account

On logging in, the ‘Login/Register’ tab will instead display user account information with the option to logout. Logging out clears all user session information. Figure 21 shows the home screen once the user has logged in, while Figure 22 shows the ‘Accounts’ tab.

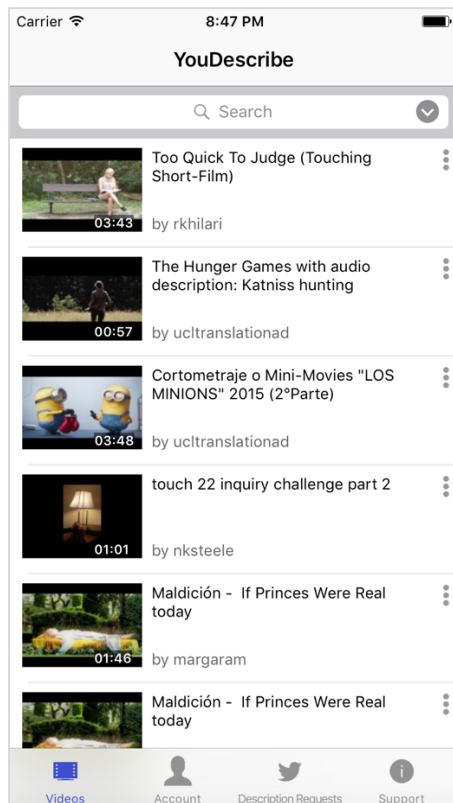


Figure 21: Home screen showing 'Account' settings once user has logged in

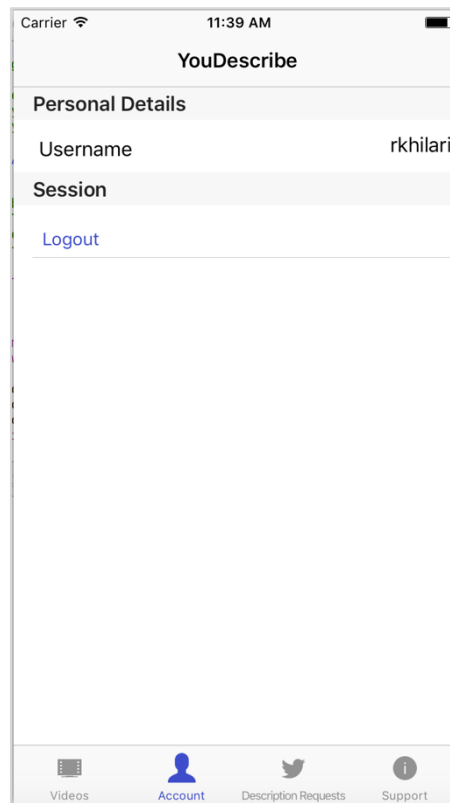


Figure 22: User account or settings scene

4.4.5. Search for videos

Entering a search query in the search bar on the home screen displays the search results for the given keyword. Described videos can be searched for by video name and/or describer. Performing a search returns the list of previously described videos followed by the non-described ones listed on YouTube. Figure 23 shows the search results on searching for the term 'Hello'.

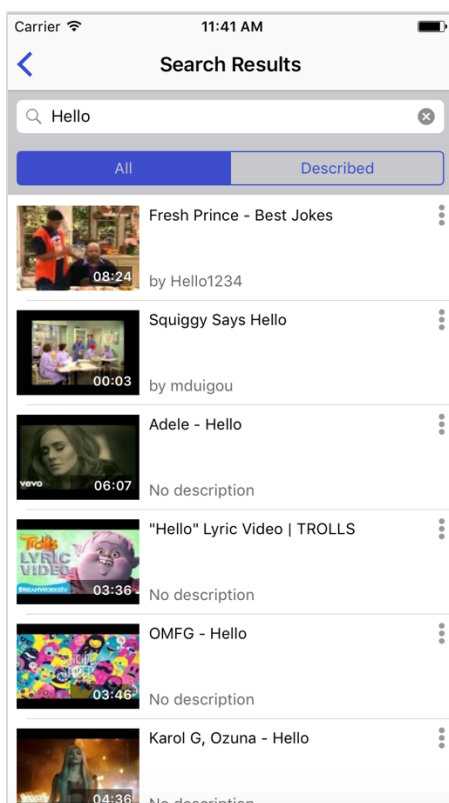


Figure 23: Search results showing 'All' and 'Described' filters for videos.

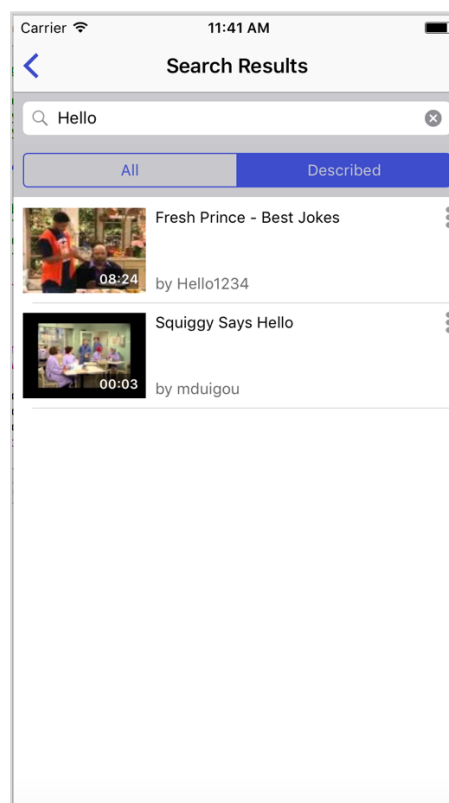


Figure 24: The search results filtered to show only matching described videos.

It is possible to view all videos described by a particular describer through the detail menu item attached to each video. The UISearchBar attached to the UINavigationController describes the 'scope' labels 'All' and 'Described'. The search functionality is performed by doing a case-insensitive, linear search for the search query as an NSRegularExpression. Figure 24 shows the video previously described on YouDescribe which contain the term 'Hello' in either the title or the describer name. As shown in

Figure 18, videos described by a particular describer can also be listed through the details menu, the result of which is shown in Figure 25.

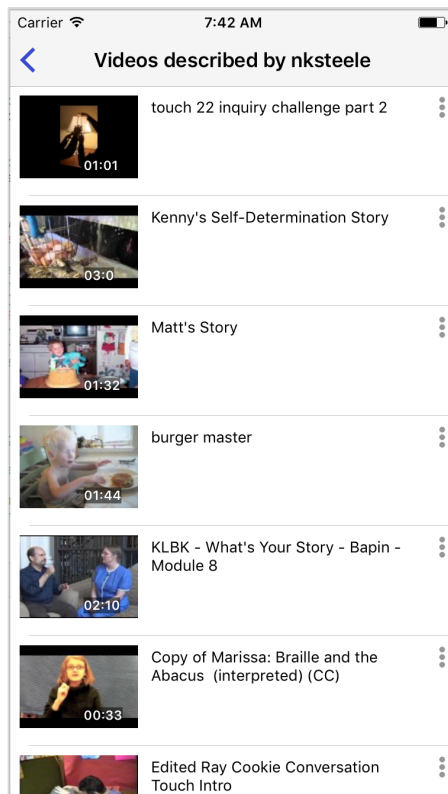


Figure 25: Videos described by a particular describer

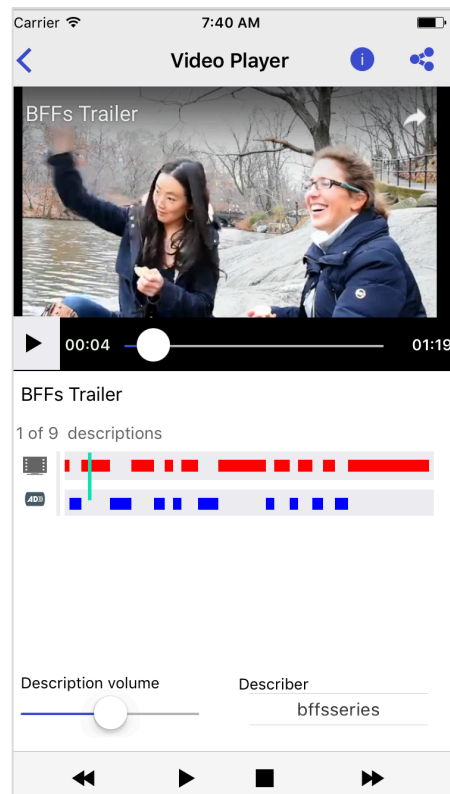


Figure 26: The player with playback controls. The user can change volume, describer, view details or share the described video.

4.4.6. Video Player Component

The player provides a set of controls that help playback the audio descriptions along with the video. It is possible to select the desired describer and adjust the volume of the

description if required. It is also possible to skip backward or skip forwards to the desired location on the video timeline. Figure 26 shows the interface of the player.

The player component uses the YouTube iOS Helper player to playback the YouTube video. It implements a `YTPlayerDelegate` that helps attach additional behaviors during events triggered at playback, state change and seeking to a position. The video player's timeline was developed to allow finer control over the video playback and accessibility experience. The set of audio descriptions associated with the selected author are downloaded and cached in temporary storage on disk.

Playback events are intercepted at approximately 0.1 seconds based on which the audio description is played at the designated start time as inline or extended clips. As the `AVPlayer` does not have built in support for stop or pause events based on a `Delegate`, notifications are registered and deregistered to keep track of audio playback. The player also displays the number of the current audio description playing, the total number of audio clips for that video as well as a timeline representation of audio clips along the video. This would help give the user a better understanding of the distribution of audio clips throughout the video and their durations.

The video details screen (shown in Figure 27) displays general information about the video and statistics such as views, likes, dislikes on YouTube and other calculated parameters such as the estimated description completeness. It uses a static `TableViewController` to display information and other YouTube and YouDescribe

statistics of the video. Currently the description completeness estimation implements the same algorithm for audio descriptions as on the web interface.

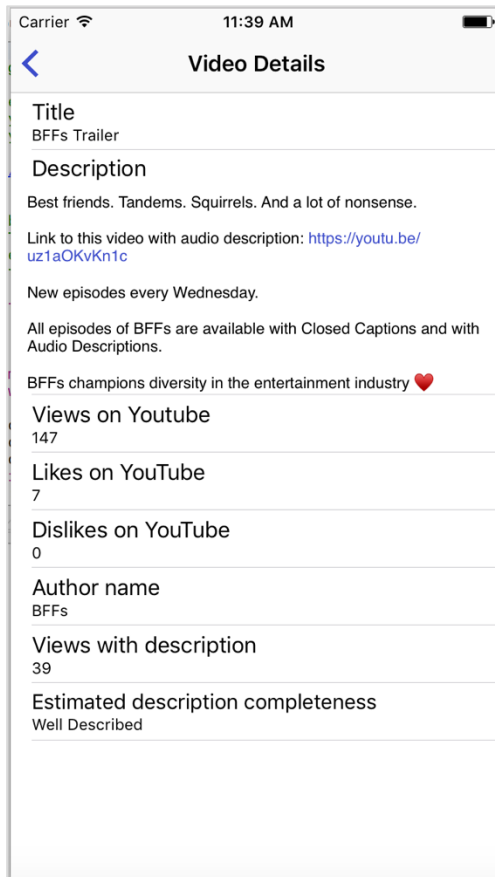


Figure 27: General information about the described video. The description completeness is estimated as well described based on the algorithm listed.

Algorithm to estimate description completeness:

```
descriptionEffort =
((lastClipStartTime -
firstClipStartTime)/totalVideoDuration) *
(numberOfClips/(totalVideoDuration/60.0))
```

```
if descriptionEffort < 0 then
    return "Unavailable"
else if descriptionEffort < 0.25 then
    return "Incomplete"
else if descriptionEffort < 0.75 then
    return "Underdescribed"
else
    return "Well Described"
```

where totalVideoDuration is the duration of the video in milliseconds.

firstClipStartTime is the start time of the first audio description

lastClipStartTime is the start time of the last audio description.

4.4.7. Sharing and Requesting for audio descriptions

The user can share one or more described videos via the social apps installed on the device such as Email, iMessage, Google+, Facebook, Twitter. A custom message with a

URL to the video described by that particular describer will be launched. It also allows users to copy a link to the clipboard, or copy HTML code to embed the player in a website. During the time this app was built, description requests were made through Twitter using the hashtag #YDRequest. Hence the app allows requesting of descriptions through social media using the same hashtag. The new API has support for a description request tracking system, visible to the user as a 'WishList' [Appendix. 3]

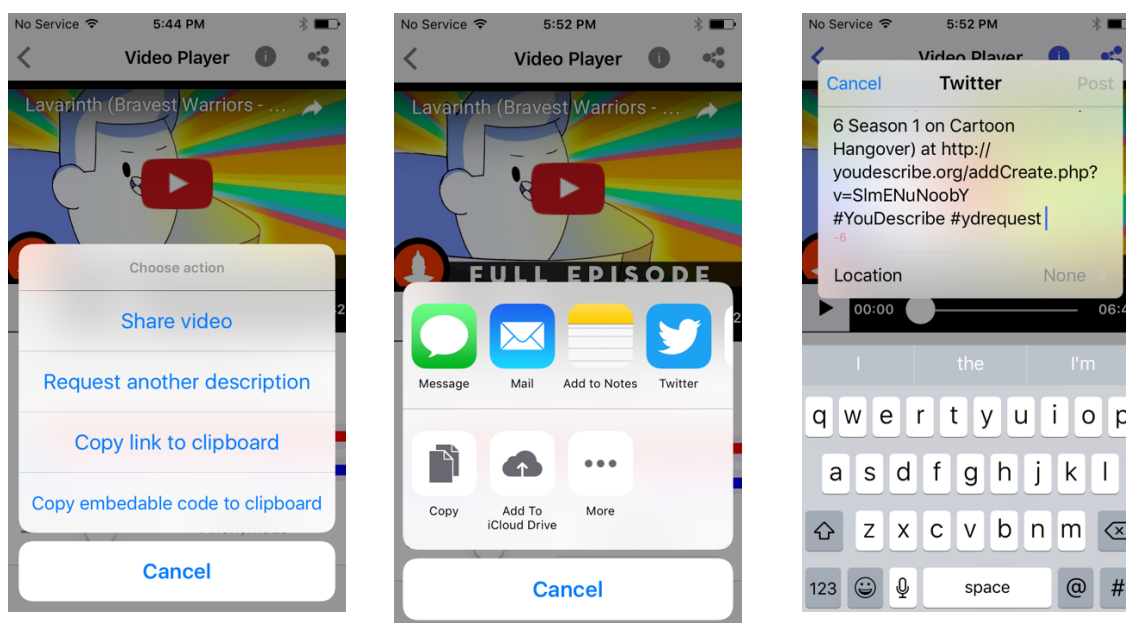


Figure 28: Shows the menu on clicking the 'Share' icon on the top right. Requesting a new description for a video shares a direct link to describe the video on the required social platform. Until now, twitter has been the platform of choice for keeping track of description request made on www.youdescribe.org.

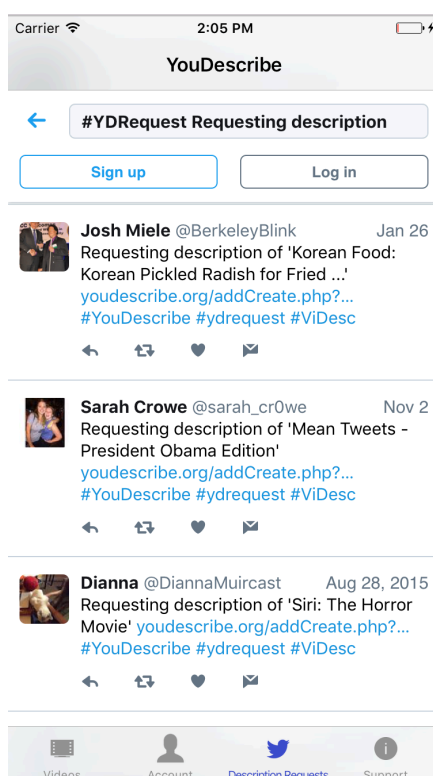


Figure 28b: Description Request tweets

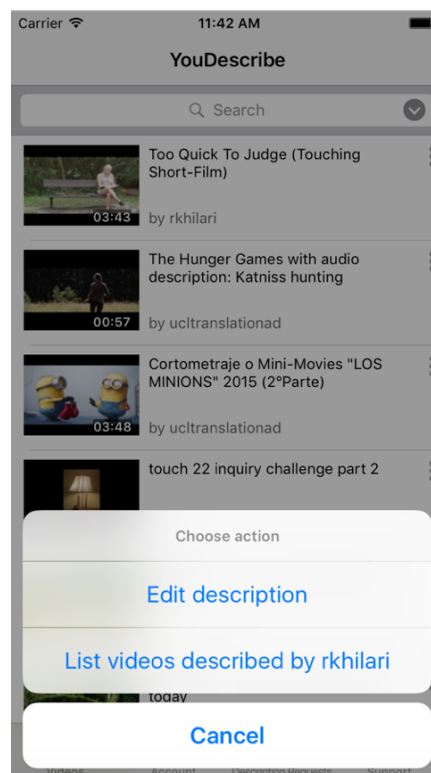


Figure 29: Option to edit description on Detail menu

4.4.8. Authoring mode

The authoring mode enables a sighted volunteer to record and save audio descriptions for existing videos. Once logged in, they can create audio descriptions for a previously described or a non-described video from YouTube. They can also edit videos that they had previously authored. The YouDescribe player is made available from within the authoring mode to allow volunteers to playback their audio descriptions as soon as they upload them. The interface shown below allows the user to play the video, record an audio clip and then review it before uploading (saving) it. Once saved, the audio clip

entry appears in the list with various options for editing its parameters (such as nudging the clip along the timeline to the left/right by a second or a millisecond, changing the clip function between inline and extended), deleting the audio clip and playing it back.

The recorder uses a `UINavigationController` to embed the player along with the audio-video timeline. Custom cells in the `UITableView` display the described clips, their start time and their properties. Recording is performed using the `AudioHelper` that directly encodes the PCM data from the `AVAudioEngine` into MP3 format.

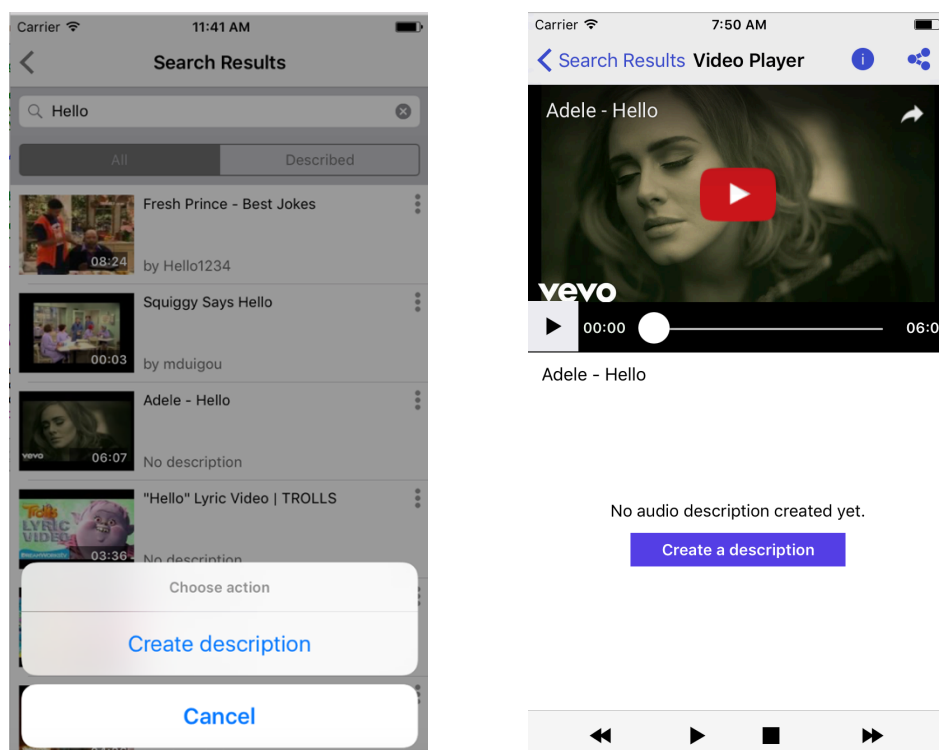


Figure 30: Option to create a new audio descriptions for videos on YouTube that are not previously described



Figure 31: Authoring (Recording) interface with an embedded audio player

4.5. Developing for accessibility

iOS includes the UI Accessibility programming interface, that is an API that helps the application provide all the information required by VoiceOver to help visually impaired people use the app. It is a part of the UIKit and hence is available through all UI Controls

available to iOS. Implementing these customizations on the UI Controls is the key to making the app accessible.

4.5.1. Accessibility attributes and classes

Accessibility attributes are various properties of UI Controls that can help make it accessible. Following attributes are defined for this purpose:

1. Accessibility Label: A short word that describes exactly what the function of the control is. It is the first element read by VoiceOver and hence should be most intuitive. E.g. “Play”
2. Accessibility Traits: A combination of one or more words that describes the current state, behavior or usage. E.g. “Button”
3. Accessibility Hint: A brief phrase that describes the result of the action. E.g. “Plays a video”
4. Accessibility Frame: Indicates the screen position and size of the element.
5. Accessibility Value: Commonly a numerical value of the control if that information is not given in the Label. Eg. For a slider, the value could be “90%”
6. Accessibility Elements: This represents the list of UI controls that will be read out by VoiceOver. It is possible to determine the order in which the labels will be accessed by VoiceOver through this attribute.

Other accessibility classes include:

1. `UIAccessibilityContainer`: This is useful to implement accessibility for certain non-view elements of a container that are not accessible by default. By implementing methods provided by the `UIAccessibilityContainer`, it is possible to make subcomponents accessible as separate elements.
2. `UIAccessibilityElement`: Objects of this class serve to encapsulate information about an item that is not accessible by default.
3. `UIAccessibilityCustomAction`: This makes it possible to perform a custom action on an accessible object.

4.5.2. Accessibility related tasks

Following were some of steps taken to make the app more accessible:

1. Helpful, concise hints and labels were assigned to accessibility labels.
2. `UIAccessibilityCustomAction` was extended to add more actions to the accessibility rotor to read out “More Actions” on a particular video item displaying an entry.
3. Standard accessibility gesture support including “Magic tap” was implemented on certain screens which required it, such as the Player and Video lists.
4. In iOS, the standard controls are accessible by default, hence the accessibility tab order is preserved for most cases.
5. It is necessary that all controls to be reachable by VoiceOver are marked as an `AccessibilityElement`, defining their order in the `accessibilityElements` attribute.

4.6. Challenges faced

Following are some of the challenges faced during the implementation of the project and the course of action taken to resolve them.

4.6.1 Challenges involving Audio/Video Player

1) *Player resolution*

A number of video players for iOS were reviewed during the start of the implementation. We first chose the YouTube iOS Helper Player, which gave a default control accuracy of 0.5 seconds. As a result of this, the video/audio synchronization with that of the audio clips was not as accurate at the web interface.

Solution: It was possible to work around this issue as the code for the YouTube iOS Helper player is Open source. On manually importing the player rather than using the built Cocoapod, the rate at which the playback event was called was modified in the source code, to achieve an accuracy of 0.1 second. No issues were observed in the expected behavior due to this change.

2) *Audio ducking*

iOS has the ability to lower the volume of the app's audio while Voiceover is talking. Once the *VoiceOver* message is completed, the volume of the app is returned to the previous level. This is known as audio ducking. Similar behavior was required within the app wherein the volume of the video's soundtrack should

‘duck’ when an inline audio clip is being played back. According to tests with AVAudioSession, it seems that audio ducking works only with background applications and VoiceOver/Speech. A solution for this problem is still under investigation using CoreAudio and the lower level AudioUnits to directly access the audio buffer.

4.6.2 Reliability of the DVX Server:

1) DVX Server crashing sometimes due to asynchronous audio clip downloads.

As the DVX Server is owned and maintained by Smith-Kettlewell Eye Research Institute, the DVXtest server has been a black box throughout the project implementation. The XML API on the test server provided by DVX seemed to be unreliable and sometimes crashed; it seemed that this happened during asynchronously downloading audio clips. Explicit restarts from the DVX developer were required, which did cause a break in implementation. Sufficient information about the server calls or logs was also not provided which made addressing the issue harder.

Solution: An alternative sequential download of audio clips was implemented, which can be switch by setting a bit in the code. There was also an upgrade of the DVX code. Following this the server did not seem to crash completely. Eventually the test server was decommissioned and a switch to the production server was made.

Figure 32 shows the internal server error that was noticed frequently:

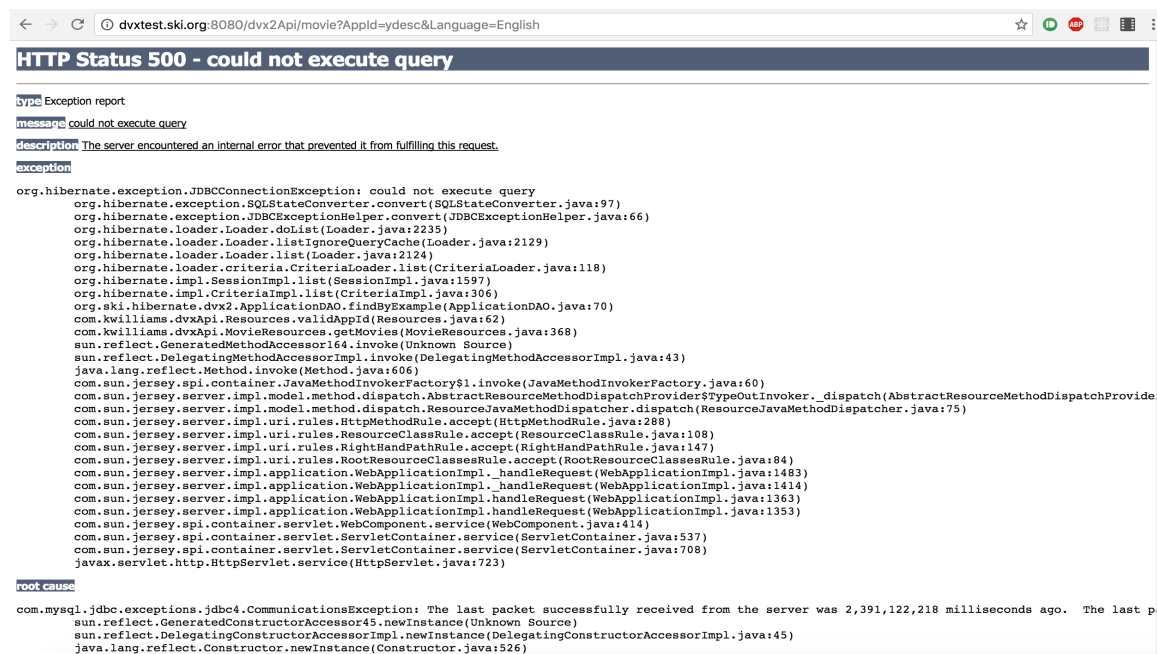


Figure 32: For a major part of development cycle, the DVX server was a black box, hence the only resolution to this was to request a service restart

2) DVX Test Server intermittently not serving the audio clips.

After the DVX server upgrade, issues with the above issue (500 server error) intermittently appeared when multiple requests were made to the server in quick succession. It was initially thought that the issue was the Alamofire library as Swift's default NSURLSession was able to download the complete clip. Once this was integrated into the application, it was noticed that this might not be the problem. The behaviour seemed non-deterministic and random as different requests would fail on each run. The error seemed to be similar to the above error. The server request pattern

and failures were studied using Charles Web Debugging Proxy^[65]. The rate of failure was reduced by adding in a delay of a couple of seconds between each call but could not be completely eliminated.

Figure 33 shows the calls to the server and the failure of calls on each run:

The following show clip download ClipId=1984 and Movie=1052 successfully downloaded in (a) and failing in (b) represented by the Bomb icon.

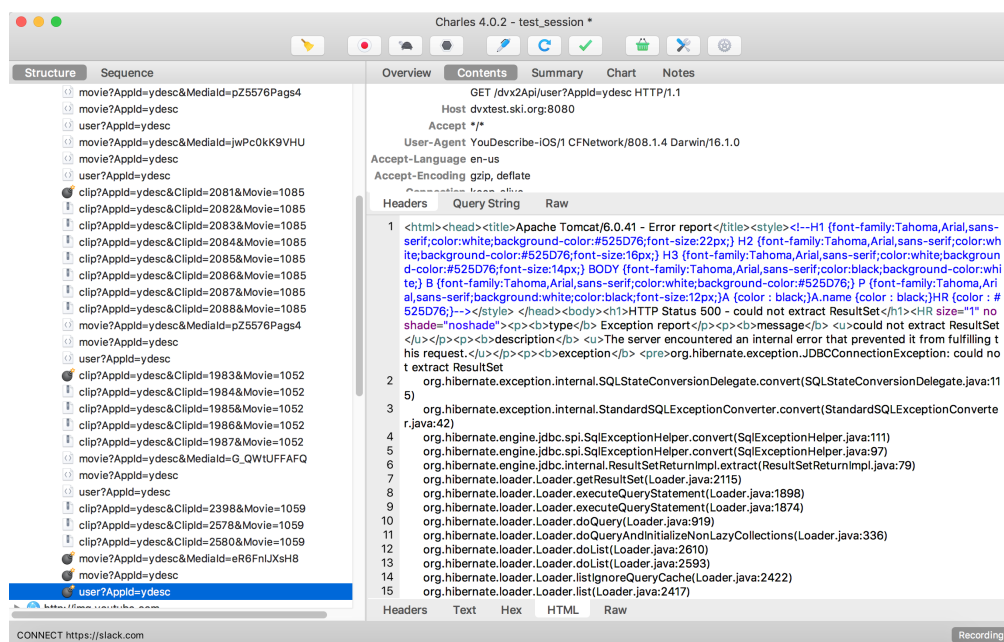


Figure 33: Sequence of requests made to the DVX Test server. Intermittent unsuccessful requests are displayed using the bomb icon.

Solution: The issue seemed to be related to the limit in the number of connections in the connection pool. It was improved by setting a `c3p0` connection parameter in the configuration file.

5.6.3. Recorder (Authoring mode)

iOS does not have native support for the creation of .mp3 files on the device. It does not support natively recording into mp3 format, as MP3 encoding is proprietary. Thus, there is no codec available on iOS for encoding mp3. AVAudioRecorder provides the AudioFormatMPEGLayer3 option but that is only for playback of a recorded file. Other audio format options included .aac and .wav; however, these formats would be inconsistent with the existing system.

Solution: Use of C++ library LAME (LAME Ain't an MP3 Encoder) allowed writing out bytes read by the AVAudioEngine into an mp3 buffer which was then written out to the filesystem. The library was included using a bridging header.

Chapter 5

Testing

This section gives an insight into the procedures adopted to test the YouDescribe iOS app.

5.1. Unit testing

Units are defined as the small pieces of testable code. Unit tests are small, isolated and independent tests run to verify the behavior of a specific functionality within the code and ensure that the code behaves as expected. For Swift 3.0, unit tests can be added through the XCode Test Navigator. It uses the XCTest framework that allows developers to create, run unit tests, performance tests and UI Test for XCode projects. On creating a

Test Target, which is usually the project itself, it is necessary to create a new test case that is a subclass of `XCTestCase`. `XCTestCase` is a subclass of `XCTest` that provides methods to override that hook into the framework such as `setUp()` and `tearDown()`. Like other testing frameworks, `setUp()` performs setting up of a initial state required for running the test case, while `tearDown()` function to perform cleanup after each

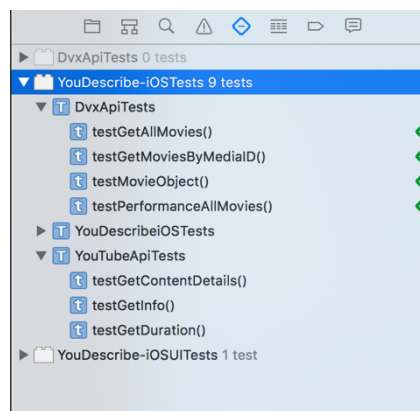


Figure 34: Unit tests for the DVX API calls

test method completes. `XCTAssert` methods are used to verify that the required conditions are always true. Figure 35 shows some of the unit tests to test calls to the DVX API.

```

9  import XCTest
10
11  @testable import YouDescribe_iOS
12
13  class DvxApiTests: XCTestCase {
14
15      var dvxApi: DvxApi!
16      override func setUp() {
17          super.setUp()
18          dvxApi = DvxApi()
19      }
20
21      override func tearDown() {
22          super.tearDown()
23          dvxApi = nil
24      }
25
26      // Tests query to get all movies
27      func testGetAllMovies() {
28          let results = dvxApi?.getMovies([:])
29          XCTAssertGreaterThan(results!.count, 1500, "Did not find more than 1500 movies")
30      }
31
32      // Test query to get movie by media ID
33      func testGetMoviesByMediaID() {
34          let results:Array<AnyObject> = (dvxApi?.getMovies(["MediaId" : "kPsyxl1aEFI0"]))!
35          XCTAssertEqual(results.count, 1, "Could not find a movie with the given MediaId")
36      }
37
38      // Tests the number of attributes returned by a movie object
39      func testMovieObject() {
40          let result:Array<AnyObject> = (dvxApi?.getMovies(["MediaId" : "kPsyxl1aEFI0"]))!
41          XCTAssertEqual(result[0].count, 7, "Mismatch in the number of attributes in a Movie object")
42      }
43
44      // Tests the performance of the query to get all movies.
45      func testPerformanceAllMovies() {
46          self.measure {
47              _ = self.dvxApi?.getMovies([:])
48          }
49      }
50  }

```

Figure 35: Units tests for DVX API functions

5.2. Usability testing

Usability testing is the process of evaluating the ease of use of a product to validate if it is usable by its representative users.

5.2.1. Usability test plan and results

A Usability Test Plan focuses on the testing one or more features for its ease of use. It is performed to ensure that the user interface and functions are intuitive to a user. The user should be able to use the app without any prior training. We pick a sample of one test plan to carry out usability testing on one of the features.

- Test objectives: To verify the usability of the player component accessed through the search function.
- User: A sighted volunteer was selected for testing.
- Test plan: Derived from the use cases, instructions for this test plan were as follows: “Search for videos from the author ‘bffsseries’. Play the first video. Display information about the video and request another description via message.”
- Intended users: A sighted volunteer tested the application.
- Conclusion: The volunteer could search for the required videos, play the video and audio descriptions from the first result. He could view the information associated with it but did not find the ‘Request Description’ option under the ‘Share’ menu intuitive. This shall be addressed in the future as a new request description system (Wishlist) would need redesigning of the UI.

5.2.2. Accessibility testing

Accessibility testing is performed to ensure that the app is usable by people with disabilities (visually impaired in this case). It can be debugged on the simulator with some limitations using the accessibility inspector. However, complete accessibility testing was performed using VoiceOver. The accessibility inspector is a tool packaged with XCode that allows one to test accessibility properties of various elements and their hierarchies. It can be accessed from the XCode Menu > Open Developer Tool > Accessibility Inspector. Figure 36 shows the accessibility labels and traits assigned to the ‘Description Requests’ tab button.

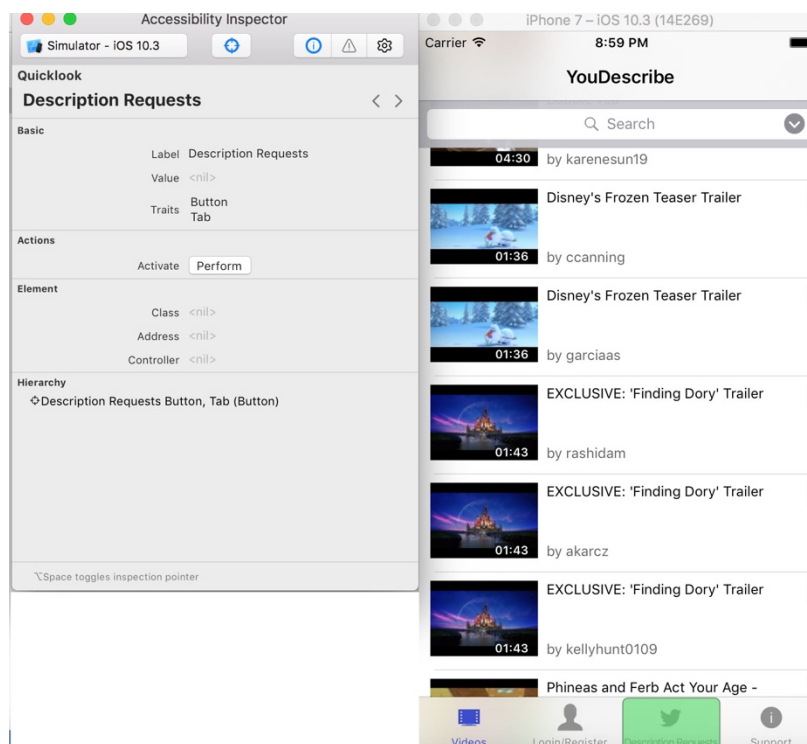


Figure 36: Accessibility Inspector showing label, traits, value for the 'Request Description' Tab button

The accessibility features of the app were also debugged using the ‘Run Audit’ button, which lists warnings in case XCode encounters potential accessibility issues. In addition, testing the app using VoiceOver gives a true experience of its interaction with a visually impaired user. Dr. Miele, the Director of R&D at Smith-Kettlewell, is a visually impaired user, and gave me first hand feedback about various aspects of accessibility required by the app. Figure 37 shows evaluation of accessibility of the home screen. On running an audit through the Accessibility Inspector on the home screen, it warned us of an element that has not description for accessibility support. This control was later removed as it had no function yet.

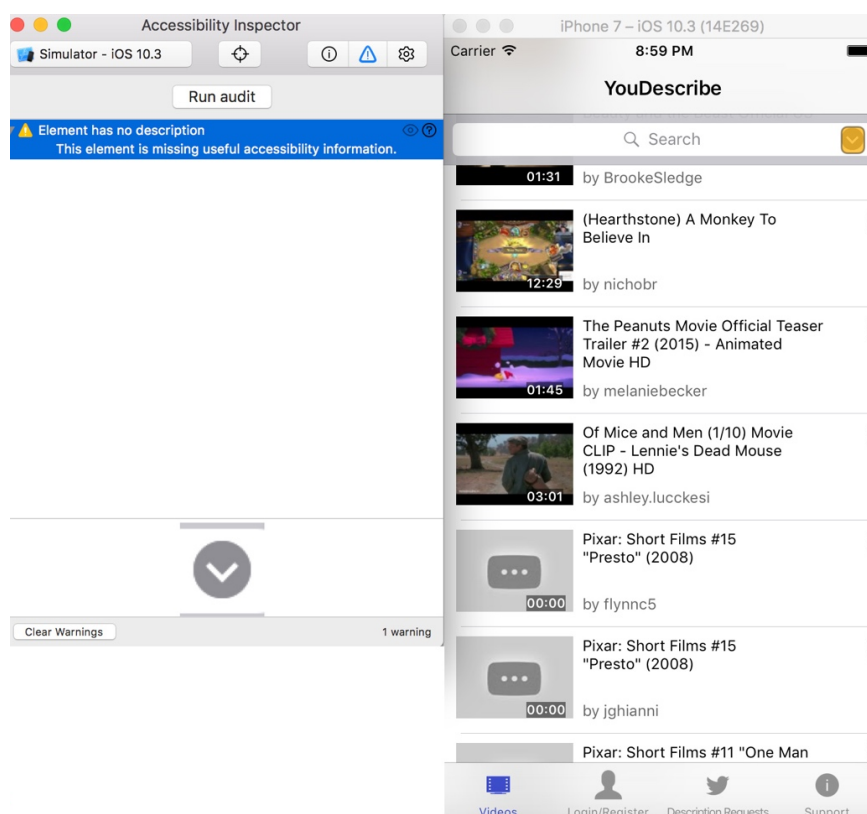


Figure 37: Accessibility Inspector showing missing description for button shown on ‘Run Audit’

Some feedback on accessibility included:

1. Magic tap gesture should be included in the player to toggle play/pause.
2. The 'Details' menu for each video item should not be explicitly read out loud according to accessibility standards.

The above feedback was taken into consideration and implemented. (2) was implemented by extending the `UIAccessibilityExtendedAction` to produce the desired outcome. Figure 38 shows the text spoken by VoiceOver on getting focus.

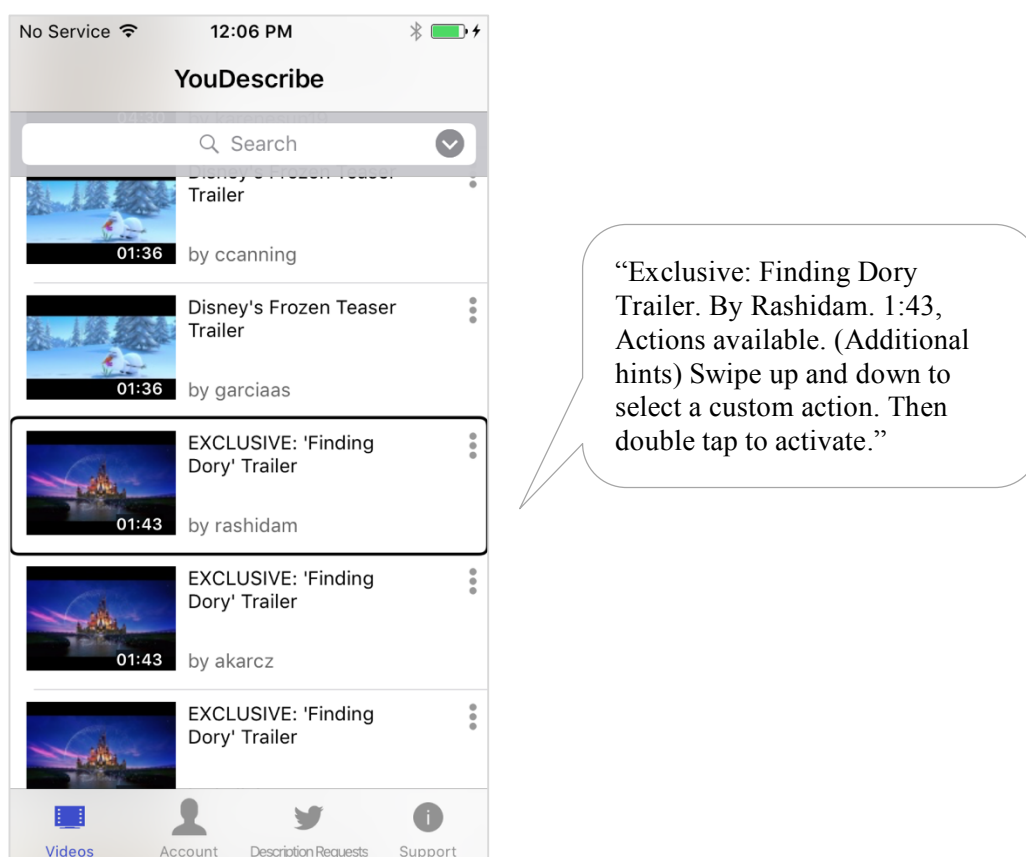


Figure 38: Text spoken by VoiceOver on navigating to the element through gestures

5.3. Quality Assurance (QA) testing

A Quality Assurance Test Plan is a document that outlines the objectives, scope, setup, approach and focus of any software testing effort in terms of its functionality. It can be described with the help of the following sections:

- Test objectives: To perform manual testing on all UI elements to determine if each functions as expected.
- Hardware and software setup: The app was tested through installation via XCode on an iPhone 5. Given below is a list of possible test cases, its input, expected output and results.

Table 4: Functional tests for the YouDescribe iOS app

No.	Test name	Description	Input	Expected output	Result
1.	Search by video name	Search for one or more videos by author from the home screen.	Enter 'BFF' in the search bar.	3 described videos show up along with 20 more YouTube results.	The result is as expected
2.	Search by author	Perform search from the search results screen for an author	Enter 'pixiediesel' in the search bar, hit search.	Display 22 videos as search results along with 30 more YouTube video items	The results are as expected
3.	Play video with inline	Make sure inline audio	Select on "Too quick to judge	Within the first 10 minutes 2 inline	The result is

	descriptions	descriptions are audible with the video	by rkhlari”	description should play and be audible	as expected.
4.	Play video with extended descriptions	Make sure the extended video clips are audible	Select on “touch 22 inquiry” by nksteel	One extended description plays back at time 00:00.	The result is as expected.
5.	Toggle between describers while video is playing	Change the describer from the list of describers while the movie is playing.	Play the video ‘Two chips’ by author pixiediesel. Switch to author kultj	The user get a prompt about restarting the video if the author is changed. On confirming, descriptions by the chosen author are played.	The result is as expected.
6.	Audio volume change	Increase and decrease the describer audio volume while the description is playing	Play the above video, increase and decrease the audio level using the volume slider.	The describer’s audio volume increases and decreases based on the slider value.	The result is as expected.
7.	Skip forward during playback	Click the skip forward button during video playback.	Play the above video, click the skip forward button at 00:03	The audio description being played stops, the video skips 10 seconds and resumes playing.	The result is as expected.

				The following description plays.	
8.	Skip backward during playback	Click the skip backward button during video playback.	Play the above video, click the skip backward button at 00:13	The audio description being played stops, the video moves 10 seconds behind and resumes playing.	The result is as expected.
9.	Request for audio description	Request description by sharing on social media from the player scene.	Hit the 'Share' icon on the top-right of the player scene, click on request description, selected the Email app.	As Email is configured, this action will open a new email with an appropriate message beginning with 'Requesting description..'	The result is as expected.
10.	Display statistics about a video	Display video details and statistics from the player scene.	Hit the 'Info' icon on the top right on the player scene	A new scene with YouTube and YouDescribe statistics is displayed.	The result is as expected.
11.	Create a new audio description	Add a new audio description clip.	Record a new audio clip at time 00:50, preview and	After recording, the describer should be able to preview the clip	The result is as expected.

			upload it.	and upload it at 00:50	
12.	Edit the audio description	Edit one of the properties of the audio description	Nudge the clip's start time by 0.1 seconds to the right by clicking on the '>' button	The new start time is now displayed on the label above the audio playback slider.	The result is as expected.
13.	Delete the audio description.	Remove an existing clip from an audio description.	From the list of audio descriptions, delete the test added in (11)	The audio description should be deleted after confirmation from the describer.	The result is as expected.

5.4. Performance testing

The iOS app was tested for performance with regards to memory consumptions, CPU cycles and load. This can be done with the help of the Instruments app^[51] by Apple. It is a performance analysis testing tool packaged as a part of XCode's toolset. It allows the app to be tested based on different parameters such as disk I/O, memory usage, memory leaks, network usage, etc. The test was performed on an iPhone 5, running iOS 10.3.1, network speed of about 16 Mbps download and 1.30 Mbps upload. Statistics about several activities were collected while the app was launched. Following this, 'Top 3 best eagle attacks', a video that was not cached on the device was selected for playing. Figure 39 is an overview of the findings for a duration of 1 minute 10 seconds of runtime.

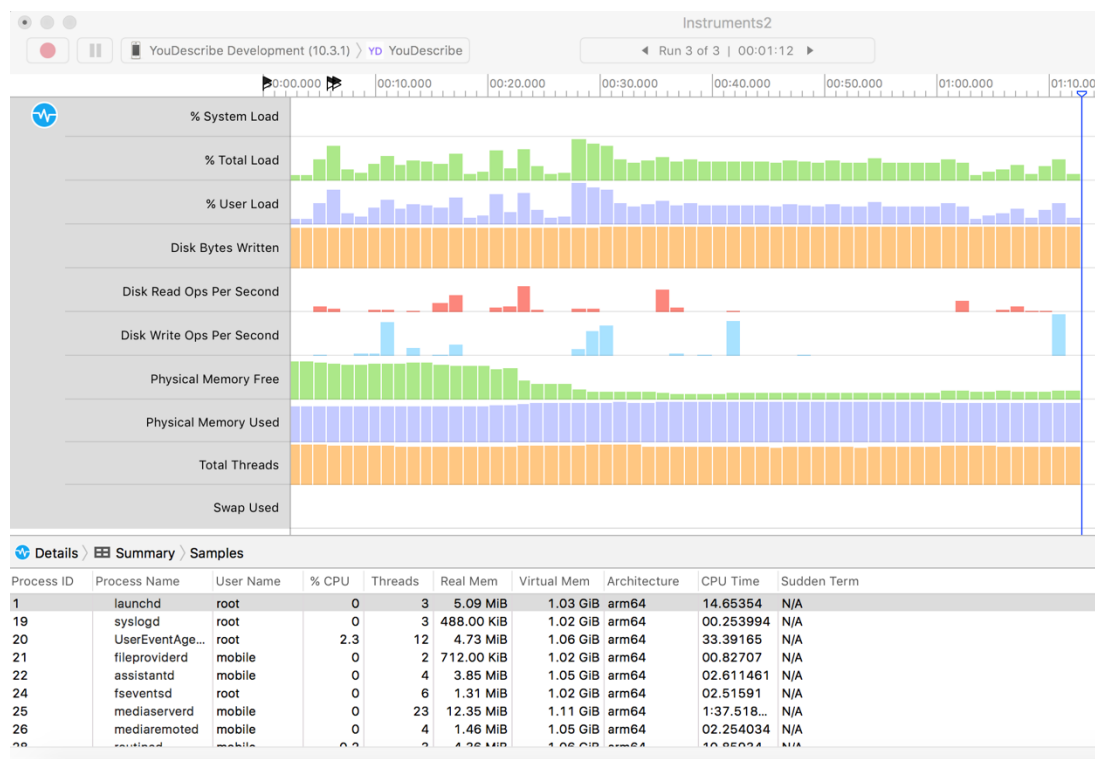


Figure 39: Screenshot of the Instruments app while beginning to play a described video through the iOS app

From the statistics, we learn that there is maximum load (82%) on the system when the user clicks on the play button initially, that is when all the audio clips belonging to that author for the video are downloaded, and cached on disk. We also notice the number of disk write ops spiking to 62 per second. Once cached on disk, the load reduces to a state of equilibrium. Following this an audio clip is read from the disk to play back resulting in an increase in the Disk read ops per second to 103. The maximum amount of memory taken by app while running was 62 MB. We observed that the memory usage recorded while running the app in the Simulator was almost double compared to the actual usage on the device.

5.5. Code review and linting

Peer review of code was not performed during the development of the project. However, static analysis was performed on the written code using a tool ‘tailor’^[52] to make sure the code adheres to specific iOS standards as given in their Swift programming style guide.

Figure 40 shows style violations in the DvxApi.swift file, while Figure 41 shows the result once the violations were fixed.

```
[2017-05-07 22:49:18][Rupals-MacBook-Pro YouDescribe-iOS]$ tailor ./YDiOSViewer/DvxApi.swift
Analyzing 1 file:
.
/Users/rupal/YouDescribe-iOS/YDiOSViewer/DvxApi.swift:10:24 warning: [colon-whitespace] Colon at column 23 should have exactly one space after it
/Users/rupal/YouDescribe-iOS/YDiOSViewer/DvxApi.swift:17:12 warning: [redundant-parentheses] Condition should not be enclosed within parentheses
/Users/rupal/YouDescribe-iOS/YDiOSViewer/DvxApi.swift:17:31 warning: [brace-style] Opening brace should have exactly one space before it
/Users/rupal/YouDescribe-iOS/YDiOSViewer/DvxApi.swift:25:16 warning: [colon-whitespace] Colon at column 16 should have exactly one space after it
/Users/rupal/YouDescribe-iOS/YDiOSViewer/DvxApi.swift:25:48 warning: [trailing-whitespace] Line should not have any trailing whitespace
/Users/rupal/YouDescribe-iOS/YDiOSViewer/DvxApi.swift:27:34 warning: [colon-whitespace] Colon at column 34 should have exactly one space after it
/Users/rupal/YouDescribe-iOS/YDiOSViewer/DvxApi.swift:30:16 warning: [colon-whitespace] Colon at column 16 should have exactly one space after it
/Users/rupal/YouDescribe-iOS/YDiOSViewer/DvxApi.swift:34:27 warning: [colon-whitespace] Colon at column 27 should have exactly one space after it
/Users/rupal/YouDescribe-iOS/YDiOSViewer/DvxApi.swift:35:16 warning: [colon-whitespace] Colon at column 16 should have exactly one space after it
/Users/rupal/YouDescribe-iOS/YDiOSViewer/DvxApi.swift:36:16 warning: [colon-whitespace] Colon at column 16 should have exactly one space after it
/Users/rupal/YouDescribe-iOS/YDiOSViewer/DvxApi.swift:41:22 warning: [forced-type-cast] Force casts should be avoided
/Users/rupal/YouDescribe-iOS/YDiOSViewer/DvxApi.swift:42:22 warning: [forced-type-cast] Force casts should be avoided
/Users/rupal/YouDescribe-iOS/YDiOSViewer/DvxApi.swift:47:15 warning: [forced-type-cast] Force casts should be avoided
/Users/rupal/YouDescribe-iOS/YDiOSViewer/DvxApi.swift:48:15 warning: [forced-type-cast] Force casts should be avoided

Analyzed 1 file, skipped 0 files, and detected 14 violations (0 errors, 14 warnings).
```

```
[2017-05-07 22:49:49][Rupals-MacBook-Pro YouDescribe-iOS]$
```

Figure 40: Code linting for DvxApi.swift using tailor linting tool

The specific rules causing the issues were reviewed using the `–show-all-rules` flag and the violations were fixed. Given below are details of the violations.

1. **forced-type-cast:** Flag uses of the forced form of the type cast operator (`as!`).
2. **redundant-parentheses:** Control flow constructs, exception handling constructs, and values assigned in variable/constant declarations should not be enclosed in parentheses.
3. **colon-whitespace:** Flag whitespace violations around colons (`:`).


```
[2017-05-07 23:38:10][Rupals-MacBook-Pro YouDescribe-iOS]$ tailor ./YDiOSViewer/DvxApi.swift
Analyzing 1 file:
.

Analyzed 1 file, skipped 0 files, and detected 0 violations (0 errors, 0 warnings).
```

Figure 41: Shows the above violations corrected

5.6. Continuous testing through TestFlight

TestFlight is a set of tools provided by Apple to allow an easy way of distributing the iOS app for testing. On installing the TestFlight app, the tester receives notification or invitation from the developer to test the app. The app provides important information such as number of sessions, crashes, logs, installations etc. It allows app developers to easily invite users through iTunes Connect and collect valuable feedback from them before the app is released on the app store. It uses two groups by default; internal testers which should consist of personnel closely involved with the development team, and external testers which would cater to a wider audience of up to 2000 users. TestFlight can be accessed by logging into <https://itunesconnect.apple.com> with a developer account. Given a preconfigured provisioning profile, an update to the app would simply require the archival and upload of the .ipa file. Following a beta app review for external testers, once approved, the testers receive a notification to update the app on their iOS devices. The YouDescribe app was released on TestFlight periodically (on an average of once every 10 to 12 days) to the project stakeholders (internal and external). The application development benefitted from their continuous feedback and testing.

The steps followed for deployment on TestFlight can be found in [53]. Once the initial setup is performed subsequent updates to the app were made available through the following steps:

- 1) Select 'Generic iOS Device' as the current device.
- 2) In the project's build settings, increment the build number or version number depending on the requirement.
- 3) Select Product > Archive to create the .ipa archive.
- 4) Once the archive is created, it will prompt the developer to export the archive. The developer can choose to export the archive for all devices and save for Ad-hoc development.
- 5) Once exported, click 'Upload to App Store' to upload the app and make it available on iTunes connect. Select the developer team's provisioning profile for code signing.
- 6) Once the upload is completed, the developer would see the app in the 'Processing' status in the TestFlight Build section of the app on iTunes Connect.
- 7) Following the Beta app review performed by Apple to verify if the app is suitable for external testing, the new build can be selected for testing via the 'Select new Build to test' button.
- 8) On saving, email and TestFlight app notifications will be sent out to all the internal and external testers.

Chapter 6

Deployment

Software deployment involves all the processes required to make the system available to its users. In the case of an iOS app, software deployment is the process of making it available in the Apple app store publically. It involves all the steps required for TestFlight deployment with a few additional steps. The app can be released on the app store to the public at present. However, as DVX has now been made obsolete, deployment to the app store for public use shall only be done once the new YouDescribe API and backend are released and the iPhone app is migrated to use this. Currently the web interface to use this API is in development.

Figure 42 shows an overview of the tasks to be performed for deploying the app to the app store. We have already outlined steps to upload the app to TestFlight in section 5.6, and submitting the app to the iStore needs a couple of more steps.

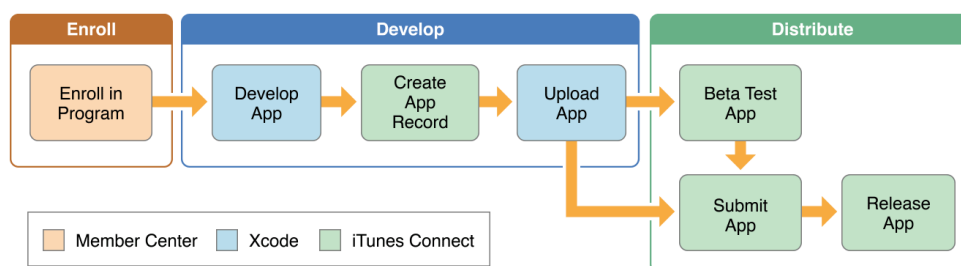


Figure 42: App deployment to the iOS App Store (Picture courtesy: Apple documentation)

We summarize the process as follows:

1. Register for a paid Apple developer account by enrolling in the Apple developer program . A developer can choose to enroll as an individual or as an organization.
2. It is necessary that every iOS app has an Apple certificate. If there is no signature or if the certificate is invalid, the app will not run on the device. Thus, on developing the app in XCode, one needs to create a profile or a code signing identity.
3. Sign into iTunes Connect to create an app record and enter necessary information.
4. In Xcode, create an archive (based on the steps outlined in section 5.6) and sign it with your distribution certificate.
5. On uploading the app using XCode, use iTunes Connect to submit the app to the store.
6. When the app is approved, use iTunes Connect to release it by setting the date when the app will be available to customers.

Chapter 7

Future Plans

The DVX backend is currently in active development and has been recently rewritten from the ground up, using a more modern technology stack (NodeJS, MongoDB, Mongoose) which is accessed through a REST API. The DVX server and the YouDescribe PHP server will soon be made obsolete and be retired. A new web interface using this new API is currently in development. With this in mind, following is a list of enhancements that are planned for the YouDescribe iOS app, some involving changes to the backend API as well.

1) Migrating to new YouDescribe API and release to app store

Apart from being rewritten with a new technology stack, the backend API is being redesigned with additional features like a description request tracking system and a feedback system that would enhance the user experience to a great deal. Keeping this in mind, our first priority would be to port all the existing functionality of the YouDescribe iOS app to use the new backend API including its additional features. Once this is done, we also plan to release the new version on the app store to make it available to the public.

2) Multiple Video sources

Going forward, multiple video sources should be supported, targeting Hulu, Vimeo and other popular video sources.

3) Porting audio descriptions

As described in the functional requirements, it would be very useful if existing audio descriptions from a describer could be ported to another online video. That way the audio descriptions could still be reused if the original video is taken down.

4) Text to Speech

A text to speech module would enable textual descriptions to be read out by the device's text reader. This could encourage volunteers with non-native English accents to contribute their descriptions as well.

5) Speech to Text

The ability to convert speech to text would help generate a textual transcript of the audio description. With this, it would be possible to search for videos using their audio description and improve the current search results.

6) Analytics

Adding analytics to the app, or integrating it with an analytics function of the website could provide additional insights about the app usage and performance in production.

Conclusion

We built the YouDescribe iOS app, which provides a novel platform for volunteers to add audio descriptions to YouTube videos to aid the blind and visually impaired users to understand the visual content in video clips. In this technical report, we have presented details about the existing system, design decisions, development process, technical aspects of developing an accessible iOS application and procedures followed for quality assurance. Our iOS application, developed in Swift 3.0, provides a complete, accessible mobile interface to all functionality offered by the YouDescribe website. This includes the ability to search for videos, playback audio descriptions over videos, request audio descriptions, and create new audio descriptions for videos. This app would enable anyone with an iOS device to contribute to or enjoy videos on YouDescribe while they are away from their computers. This would not only increase viewership but would also increase participation in the movement to create accessible content for everyone.

References

1. Audio description https://en.wikipedia.org/wiki/Audio_description ,
<http://www.acb.org/adp/ad.html>
2. MAG. <http://main.wgbh.org/wgbh/pages/mag/services/description/>
3. AFB. <http://www.afb.org/default.aspx>
4. DVS. https://en.wikipedia.org/wiki/Descriptive_Video_Service
5. YouTube . <https://www.youtube.com/>
6. Hulu <https://en.wikipedia.org/wiki/Hulu>
7. Vimeo. <https://en.wikipedia.org/wiki/Vimeo>
8. SKIERI and VDRDC <https://ski.org/> <http://www.vdrdc.org/>
9. DVX. <http://www.vdrdc.org/research/dvx>
10. YouDescribe. <http://youdescribe.org>
11. iOS. <https://en.wikipedia.org/wiki/IOS>
12. Swift. [https://en.wikipedia.org/wiki/Swift_\(programming_language\)](https://en.wikipedia.org/wiki/Swift_(programming_language))
13. HTML5. <https://en.wikipedia.org/wiki/HTML5>
14. Javascript. <https://en.wikipedia.org/wiki/JavaScript>
15. Bootstrap. <http://getbootstrap.com/>
16. JWPlayer . <https://www.jwplayer.com/>
17. Android. [https://en.wikipedia.org/wiki/Android_\(operating_system\)](https://en.wikipedia.org/wiki/Android_(operating_system))
18. Php. <http://php.net/manual/en/intro-what-is.php>
19. Java. <https://www.oracle.com/java/index.html>
20. Hibernate. <http://hibernate.org/>
21. Spring. <http://docs.spring.io/spring/docs/current/spring-framework-reference/html/orm.html>
22. JDBC. <http://www.oracle.com/technetwork/java/overview-141217.html>
23. MySQL. <https://www.mysql.com/>
24. XML. <https://www.w3.org/XML/>
25. Xcode. <https://en.wikipedia.org/wiki/Xcode>
26. OSX. <https://en.wikipedia.org/wiki/MacOS>

27. Human interface guidelines. <https://developer.apple.com/ios/human-interface-guidelines/overview/design-principles/>
28. TestFlight. <https://en.wikipedia.org/wiki/TestFlight>
29. Fluid UI. <https://www.fluidui.com/>
30. ARM. https://en.wikipedia.org/wiki/ARM_architecture
31. Git. <https://git-scm.com/> , <https://en.wikipedia.org/wiki/Git>
32. Cocoa Touch framework.
<https://developer.apple.com/library/content/documentation/General/Conceptual/DevPedia-CocoaCore/Cocoa.html>
33. Avfoundation . <https://developer.apple.com/reference/avfoundation/avplayer>
34. Cocoapods. <https://cocoapods.org/>
35. YouTube iOS Player Helper
https://developers.google.com/youtube/v3/guides/ios_youtube_helper, <https://github.com/youtube/youtube-ios-player-helper>
36. AVPlayer. <https://developer.apple.com/reference/avfoundation/avplayer>
37. Swift YouTube Player . <https://github.com/gilesvangruisen/Swift-YouTube-Player>
38. Longtail solutions. <https://www.jwplayer.com/>
39. JWPlayer HTML <https://developer.jwplayer.com/ios-sdk/> ,
<https://support.jwplayer.com/customer/portal/articles/1406725-youtube-video-embed>
40. AVAudioPlayer. <https://developer.apple.com/reference/avfoundation/avaudioplayer>
41. Alamofire. <https://github.com/Alamofire/Alamofire>
42. SwiftyJSON. <https://github.com/SwiftyJSON/SwiftyJSON>
43. Lame Encoder. <http://lame.sourceforge.net/>
44. REST API. https://en.wikipedia.org/wiki/Representational_state_transfer
45. DVXApi. <http://dvx.ski.org:8080/dvx2Api/>
46. YouTube Data API . <https://developers.google.com/youtube/v3/docs/>
47. VoiceOver. <https://www.apple.com/accessibility/iphone/vision/>
48. TalkBack. <https://support.google.com/accessibility/android/answer/6283677?hl=en>
49. Rotor. <https://support.apple.com/en-us/HT204783>
50. Gesture images. <http://www.interactiveaccessibility.com/blog/mobile-screen-reader-gestures>

51. Instruments app.
<https://developer.apple.com/library/content/documentation/DeveloperTools/Conceptual/InstrumentsUserGuide/>
52. Tailor linting tool. <https://tailor.sh>
53. TestFlight and deployment: <http://apple.co/2rANay9>
54. Flash. https://en.wikipedia.org/wiki/Adobe_Flash
55. Youtube terms and conditions. <https://www.youtube.com/static?template=terms>
<https://developers.google.com/youtube/terms>.
56. Mp3 standard doc. <http://www.mp3-tech.org/patents.html>
57. NodeJS. <https://nodejs.org/en/>
58. MongoDB. <https://www.mongodb.com/>
59. Mongoose. <http://mongoosejs.com/>
60. Vision accessibility iOS standards for development,
<https://developer.apple.com/accessibility/ios/>
61. Developing for accessibility. <http://apple.co/2qLyueQ>
62. Testing for accessibility <http://apple.co/2rQS1Zd>
63. <http://www.circleoftheblindmice.com/2016/02/11/ios-vs-android-which-works-best-with-your-visual-impairment/>
64. App Icons from <https://icons8.com/> . Required formats created using
<https://makeappicon.com/>
65. Charles Debugging Web proxy. <https://www.charlesproxy.com/>
66. Swift coding style guide. <https://github.com/github/swift-style-guide>
67. Most popular video websites May 2017. <http://www.ebizmba.com/articles/video-websites>
68. YouTube statistics 2017. <https://fortunelords.com/youtube-statistics/>

Appendix

1. Following is an example of a typical GET request and response from the DVX server.

It retrieves metadata about a clip with ID 7423 and movie ID 1552.

```
http://dvx.ski.org:8080/dvx2Api/clip/metadata?AppId=****&ClipId=7423&MovieId=1552
```

```
▼<clips>
  ▼<clip>
    <clipActive>true</clipActive>
    <clipAuthor>2251</clipAuthor>
    <clipChapter>0</clipChapter>
    <clipCreated>20140520160622</clipCreated>
    <clipDownloadCount>73</clipDownloadCount>
    <clipFilename>14006271819511.wav</clipFilename>
    <clipFunction>desc_inline</clipFunction>
    <clipId>7423</clipId>
    <clipModified>20140520160622</clipModified>
    <clipPath>dir0</clipPath>
    <clipProtocol>DVXprotocol</clipProtocol>
    <clipStartTime>0.0</clipStartTime>
    <clipType>audio</clipType>
    <languageFk>1</languageFk>
    <movieFk>1552</movieFk>
  </clip>
</clips>
```

2. Following is an example of a typical GET request and response from the New

YouDescribe API which will replace the DVX API. It retrieves metadata about the

YouTube video with ID B8BD9txkGL4.

```

{
  type: "success",
  code: 1000,
  status: 200,
  message: "Video metadata successfully retrieved",
  - result: {
    _id: "58e7cf2a46e13dfd851a202e",
    description: "",
    youtube_id: "B8BD9txkGL4",
    views: 0,
    created_at: 20170204032820,
    title: "The Hunger Games with audio description: Katniss hunting",
    legacy_video_id: 3100,
    updated_at: 20170223111830,
    - audio_descriptions: [
      - {
        _id: "58e7cf3d46e13dfd851a286d",
        status: "published",
        legacy_video_id: 3100,
        legacy_user_id: 3596,
        video: "58e7cf2a46e13dfd851a202e",
        - user: {
          _id: "58e7cf3c46e13dfd851a286b",
          name: "ucltranslationad",
          google_user_id_token: "",
          created_at: 20170129071607,
          updated_at: 20170315110223,
          legacy_user_id: 3596,
          last_login: 0,
          email: "ucltranslationad@gmail.com"
        },
        language: "en",
        created_at: 20170317000000,
        update_at: 20170317000000,
        notes: "",
        likes: 0,
        - audio_clips: [
          - {
            _id: "58e7cf9646e13dfd851a6bfd",
            audio_description: "58e7cf3d46e13dfd851a286d",
            created_at: 20170216145317,
            playback_type: "extended",
            file_size_bytes: 0,
            updated_at: 20170216145317,
            file_name: "14872855970881.wav",
            start_time: 0.612,
            legacy_video_id: 3100,
            legacy_user_id: 3596,
            label: "",
            legacy_audio_clip_id: 30885,
            video: "58e7cf2a46e13dfd851a202e",
            end_time: 0.612,
            file_mime_type: "audio/wav",
            duration: 59.33,
            file_path: "/legacy",
            user: "58e7cf3c46e13dfd851a286b"
          }
        ]
      }
    ]
  }
}

```

3. Following is the Google Data API call to obtain content detail about video with media ID k-Z8xxygd2Y

https://www.googleapis.com/youtube/v3/videos?id=k-Z8xxygd2Y&part=contentDetails&key=AI****

```
{
  kind: "youtube#videoListResponse",
  etag: "\"m2yskBQFythfE4irbTieOgYYfBU/UC-4FMShkWNnpVI8nFTeRrn-leg\"",
  - pageInfo: {
    totalResults: 1,
    resultsPerPage: 1
  },
  - items: [
    - {
      kind: "youtube#video",
      etag: "\"m2yskBQFythfE4irbTieOgYYfBU/PumK7mRkumVF3cyLOGX1leaXhdM\"",
      id: "k-Z8xxygd2Y",
      - contentDetails: {
        duration: "PT1M46S",
        dimension: "2d",
        definition: "hd",
        caption: "true",
        licensedContent: true,
        projection: "rectangular"
      }
    }
  ]
}
```